

Vehicle Recovery & Roadside Assistance Platform

MA7443 MSc Data Science Research Project

Group Final Report

Group Members:

Name	ID
Theo Keetile	tk344
Baji Baba Shaik	bbs8
Hermann Deudjui Siewe	hads2
Abdul Kaif Mohammed	akm65

University Supervisor: Dr Eleanor Lingham

Marker: Dr Dawei Hao

Industrial Supervisor: Eliyas Woldegeorgis



**School of Computing and Mathematical Science
University of Leicester**

Date: 28 August 2026

AI Use Declaration

The authors declare that any use of generative artificial intelligence tools in preparing this report was limited to responsible academic support, such as improving clarity, structure, language, planning, or technical understanding.

All intellectual work, analysis, decisions, interpretation, and final content remain the responsibility of the named group members. Any AI-assisted output was reviewed, verified, and edited by the authors before inclusion.

No artificial intelligence tool was used to fabricate data, generate false references, misrepresent sources, or replace the authors' own academic judgement.

Individual Contribution Summary

Student 1 - Hermann Deudjui Siewe (hads2)

Main Areas of Contribution - Dispatch engine and optimisation core: KNN single dispatch, Hungarian batch dispatch, decline/timeout cascade, priority-weighted cost matrix, distance capping, backend restructuring and Railway deployment.

Role in Shared Work - Owned the algorithmic backend and its verification notebook, co-owned deployment and integration.

Evidence - dispatch.py, _dispatch_new_request / _dispatch_batch_and_notify, Verify_Dispatch.ipynb, dispatch test suite (Appendix B), Concurrency_Safety_Bug_And_Fix.ipynb.

Student 2 - Theo Keetile (tk344)

Main Areas of Contribution - Front-end feature development across both user and assistance applications covering the cancellation and fee policy, the fare and earnings views, the scene-photo capture and quick-select request intake, the SOS emergency feature, the post-job rating, the request-history and job-summary surfaces, bilingual localisation and Ethiopian phone validation.

Role in Shared Work - Owned the client-side build in close collaboration with the backend development, focusing on user experience.

Evidence - cancellation.py with its seventeen-test suite (Appendix D), the fare and earnings views, the photo-capture and quick-select intake screens, the post-job rating view, job_summary_view.dart across both apps, ethiopian_phone_validator and the SOS screens.

Student 3 - Abdul Kaif Mohammed (akm65)

Main Areas of Contribution - Bidirectional live location tracking across both applications: WebSocket broadcast of the operator's position on job acceptance and at periodic intervals, reactive polyline route rendering and map-marker updates on the driver side and ETA design using the Google Distance Matrix.

Role in Shared Work - Owned the real-time client experience on both the driver and operator sides.

Evidence - assistant_ws_view_model.dart and location_view_model.dart (operator app), assistance_view_model.dart, location_view_model.dart, waiting_view.dart and location_view.dart (driver app).

Student 4 - Baji Baba Shaik (bbs8)

Main Areas of Contribution - Rating and review system.

Role in Shared Work - Owned the trust/feedback features.

Evidence - Rating/review views and endpoints

N.B. One member (bbs8) left the project during development. The remaining members completed rating and review, originally in that slice (client rating surfaces: tk344; operator reviews dashboard and related endpoints as delivered in Appendix C).

Executive Summary

RoadAid is a deployed, end-to-end vehicle-recovery and roadside-assistance platform for Addis Ababa, built on the open-source Depannini codebase (Depannini Team, 2025). It comprises a Django Channels backed by PostgreSQL and two Flutter applications for drivers and operators, connected in real time over WebSockets. The central data-science contribution is an evaluated operator-assignment engine that treats dispatch as an optimisation problem rather than a first-come heuristic. K-Nearest-Neighbours serve a single request, while near-simultaneous requests are assigned jointly by the Hungarian algorithm to minimise total, priority-weighted travel distance, protected for the most severe cases by a critical-bypass rule and kept resilient by a decline-or-timeout cascade. A verification notebook confirms the assignment is globally optimal and across two hundred randomised scenarios, never worse than greedy dispatch. Around this core, the platform delivers the full service journey of request, pricing, matching, live tracking, completion, payment and rating, with a transparent cancellation policy, a bilingual English and Amharic interface, and optional photo and voice-note attachments. The whole system is deployed on Railway and reachable from any network.

Table of Contents

Cover Page	1
AI Use Declaration	2
Individual Contribution Summary	3
Executive Summary	5
List of Figures	7
List of Tables	8
List of Abbreviations	9
1. Introduction, Project Context and Objectives	10
1.1 Problem Definition	10
1.2 Project context and stakeholders	11
1.3 Aim and objectives	12
2. Background and Related Work	14
2.1 The assignment problem	14
2.2 Dispatch in ride-hailing and logistics	15
2.3 Prioritisation and multi-criteria weighting.....	16
2.4 Real-time delivery and the distance model.....	16
3. Data, Resources and Project Inputs.....	18
3.1 Operational and derived data.....	18
3.2 Evaluation data	20
3.3 Resources, tools, and assisted development	20
3.4 The Depannini base.....	21
3.5 Constraints and ethical considerations	22
4. Methodology and Approach	23
4.1 Dispatch as a two-mode optimisation	23
4.2 Priority-weighted cost matrix	26
4.3 Critical bypass	27
4.4 Decline cascade, reconnection and fallback.....	27
4.5 Pricing and cancellation	27
4.6 Evaluation strategy.....	29
4.7 Transactional integrity and concurrency	29
4.8 Estimated time of arrival and mapping.....	29

5. Implementation, Results and Evaluation	31
5.1 System architecture and request lifecycle	31
5.2 Dispatch engine: results	31
5.3 Priority model: results	33
5.4 Commercial and resilience features: results	33
5.5 Accessibility, intake and localisation: results	34
5.6 Requirements coverage	34
5.7 Threats to validity	36
6. Discussion.....	38
7. Limitations and Future Work	40
8. Conclusion	42
References	43
Appendices	46
Appendix A - End-to-End Job Walkthrough (Screenshots)	46
Appendix B - Dispatch Verification	52
Appendix C - Operator Earnings, Fee Settlement and Reviews	59
Appendix D - Cancellation Policy and Fee Model.....	61
Appendix E - Priority Model and Resilience Evidence	65

List of Figures

Figure 1 - End-to-end service journey across the driver app, the backend dispatch engine and the operator app.	11
Figure 2 - High-level system architecture: two Flutter clients, a Django Channels backend with the dispatch engine and a PostgreSQL store.....	12
Figure 3 - Six objectives mapped to the delivered components of the platform that realise them.	13
Figure 4 - Greedy versus optimal batch assignment; greedy follows arrival order and returns 11, while joint optimisation returns 5.	15
Figure 5 - Data model. A driver submits many jobs; an operator is assigned to many jobs over time. Green box: values computed at dispatch.	19
Figure 6 - V-model development lifecycle: each design level on the descending arm is checked by a matching test level on the ascending arm.	23

Figure 7 - Dispatch decision flow: a single request uses KNN and a batch is bypassed for critical cases, solved by the Hungarian algorithm, with a Hungarian fallback and a cascade.	24
Figure 8 - A worked batch of three requests and three operators, where the Hungarian assignment minimises the total distance.....	25
Figure 9 - The priority-weight model: triage inputs yield a tier and a score, interpolated into a multiplicative weight that amplifies the request's row.....	26
Figure 10 - The cancellation decision model: free before acceptance, within the grace window or on an operator cancellation and otherwise a capped distance-aware fee. .	28
Figure 11 - Hungarian versus greedy dispatch across 200 random conflict scenarios; every point lies on or below the equal line, so the assignment is never worse than greedy.	32
Figure 12 - Appendix Figure 1 - which operator each request is assigned to, by weighting regime. Request P (red) is stuck with the far operator under both the unweighted baseline and the rejected discount theory. Only the production amplify weighting flips it to the near operator.	68

List of Tables

Table 1 - Data inventory: raw inputs, values derived at dispatch and records retained afterwards.	19
Table 2 - Technology stack and the responsibility of each part	21
Table 3 - The main methodological decisions with the reasoning and the trade-off accepted for each.....	30
Table 4 - Requirements coverage, with the priority, status and evidence for each.....	35
Table 5 - Principal risks identified during development and how each was mitigated. .	36
Table 6- Appendix Table 1 - Summary of the evidence presented in this appendix. Full detail, including the raw output each row is drawn from, follows in B.1-B.7.	53
Table 7 - Appendix Table 2 - The constants below are the values compute_cancellation_fee() and flat_base_for_stage() are built from, taken directly from core/utils/cancellation.py.....	61
Table 8 - Appendix Table 3 - Computed live from core/utils/cancellation.py. Examples 5-9 correspond to the PerKmUpliftTests and FeeCapTests assertions in D.3.	62
Table 9 - Appendix Table 4 - Summary of test_cancellation.py's five test classes.	63
Table 10 - Appendix Table 5 - Priority and weighting infrastructure definitions.	65
Table 11 - Appendix Table 6 - Continuous score definitions. Computed live from core/utils/dispatch.py (derive_priority_tier, compute_priority_lambda, weight_from_lambda).	66
Table 12 - Appendix Table 7 - Amplification vs discounting	67

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ESI	Emergency Severity Index
ETA	Estimated Time of Arrival
ETB	Ethiopian Birr
GPS	Global Positioning System
GSMA	GSM Association (global mobile-industry body)
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
KNN	K-Nearest Neighbours
MSc	Master of Science
ORM	Object-Relational Mapping
SOS	In-app emergency distress request
SQL	Structured Query Language
WHO	World Health Organization

1. Introduction, Project Context and Objectives

1.1 Problem Definition

Roadside breakdowns in Addis Ababa are currently resolved through informal and largely opaque channels. A stranded driver typically telephones a familiar garage, calls an acquaintance or attempts to flag down passing help. This arrangement carries several structural weaknesses. There is no visibility of which capable provider is nearest, so the response depends on who happens to be reachable rather than on who can arrive soonest. No price is agreed before service, creating scope for disputes and overcharging. There is also no mechanism to distinguish a routine request from a genuinely urgent one, such as a vehicle stranded in a live traffic lane, a driver travelling with young children, or a passenger requiring medical attention. Once help has been dispatched, accountability is also limited because there are no record links to the request, the provider, the price, or the outcome. Ethiopia has one of the world's highest road-traffic fatality rates (World Health Organisation, 2018), so a slow or misdirected response to a roadside emergency carries real safety consequences.

More specifically, the core problem this project addresses is decision support for operator dispatch. Given a continuous stream of assistance requests and a pool of mobile operators with differing capabilities and positions, the system must decide which operator serves which request, and in what order, so that the response is fast, fair and appropriately prioritised. The decision must also be transparent, its quality must be evaluated rather than assumed and the result must be delivered in real time to inexpensive mobile devices operating over an intermittent network. Understood in these terms, the task extends beyond the construction of an application. It is a matching and prioritisation problem with a clear operational objective: minimising response distance under a set of real-world constraints. This is the classical assignment problem, for which a polynomial-time solution has existed since the work of Kuhn (1955) and Munkres (1957). The same matching problem sits at the heart of modern ride-hailing platforms, where assigning orders to drivers is treated as a large-scale optimisation task rather than a greedy heuristic (Xu et al., 2018). Figure 1 situates this decision within the wider service the platform delivers, from the driver's request through matching to completion, payment, rating and history.

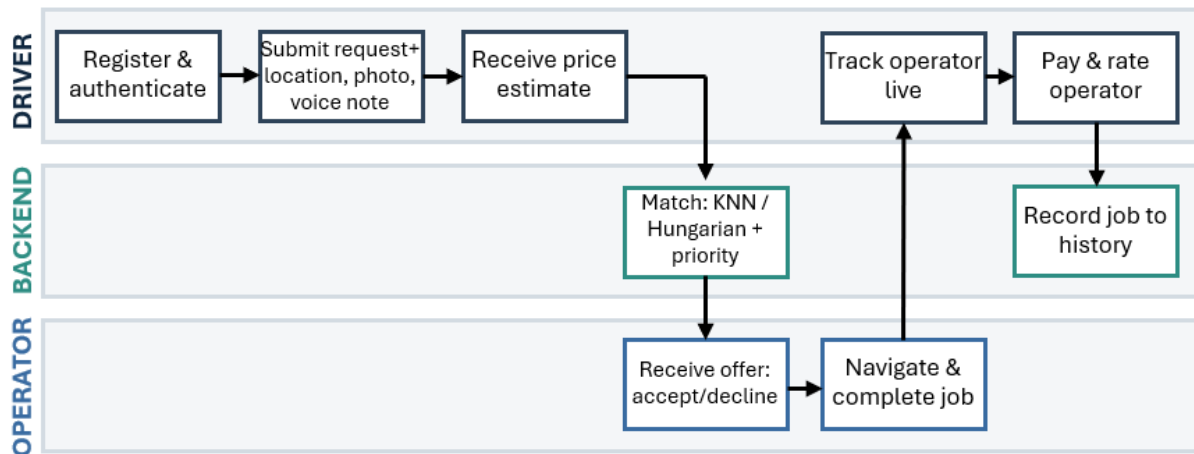


Figure 1- End-to-end service journey across the driver app, the backend dispatch engine and the operator app.

1.2 Project context and stakeholders

The project was undertaken as an MSc Data Science Research Project in collaboration with an industry client, AB Analytics Ltd, and was situated explicitly within the Ethiopian operating context. Three groups of stakeholders shape its requirements. Drivers require a fast, understandable and trustworthy way to get help and know in advance what it will cost. Operators require a fair and steady flow of jobs, clear earnings and settlement information, and protection from being penalised for circumstances outside their control. The platform owner requires an auditable record of activity and a transparent commercial model including a platform commission.

The Ethiopian context introduces constraints that shape the platform's design. Mobile connectivity is intermittent and smartphone penetration remains comparatively low (Kemp, 2024; GSMA, 2024), so the system cannot assume a continuously available real-time channel and must degrade gracefully when that channel is lost. No live card-payment gateway was available for integration, so payment is demonstrated within the application and settled at the point of service by cash, card or mobile money. The user base is bilingual, requiring the interface to be presented in both English and Amharic. Finally, Ethiopian telephone numbering, namely the +251 country code and mobile numbers of nine digits beginning with 9 or 7, must be respected in registration and authentication. These constraints are reflected in the methodology through specific design decisions. Figure 2 gives a high-level view of the resulting system: two Flutter Android applications, a Django Channels backend that houses the dispatch engine and a PostgreSQL store, connected in real time over WebSockets (Fette and Melnikov, 2011) and deployed on Railway.

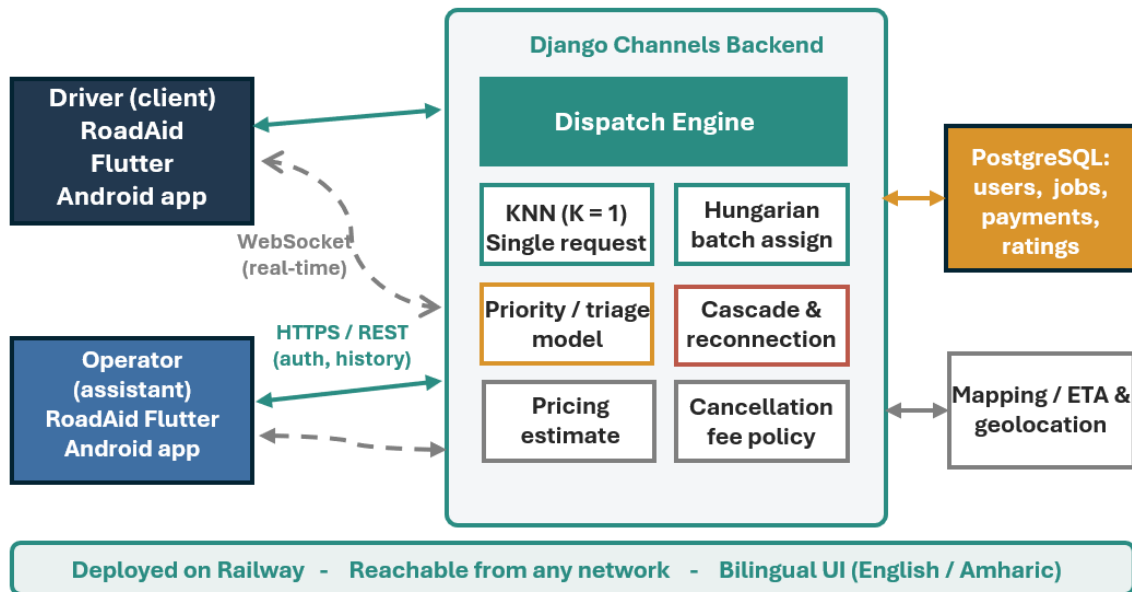


Figure 2 - High-level system architecture: two Flutter clients, a Django Channels backend with the dispatch engine and a PostgreSQL store.

1.3 Aim and objectives

The aims raise three broad research questions, addressed in turn throughout the report.

The first is whether simultaneous requests can be assigned to operators as an exact optimisation problem, producing a measurably better allocation than a simple nearest-operator allocation rule, without adding an unreasonable delay.

The second is whether a severity signal, built from what the client reports, can be folded into that assignment so the most urgent requests are protected, while the rule stays easy to understand and difficult to game.

The third is whether the resulting system holds up where it needs to, made resilient to a poor connection, commercially fair, accessible in two languages, and properly deployed, given the constraints of the Ethiopian market.

Answering these questions required six concrete pieces of work summarised below and mapped onto the parts of the platform that deliver them in Figure 3.

- **Matching** - assign operators to requests to minimise total travel distance using an exact method for simultaneous requests, with the improvement demonstrated rather than assumed.
- **Prioritisation** - derive a severity signal from what the client reports and build it into the assignment so urgent requests are protected without the mechanism becoming opaque or easy to abuse.

- Resilience - keep the platform working through dropped connections, declined jobs, timeouts and missing location data.
- Commercial fairness - provide transparent pricing, a defensible cancellation policy, operator earnings and a rating system.
- Accessibility - deliver a bilingual English and Amharic interface that lets a customer add a photo and a short voice note to a request.
- Deployment and evidence - deploy the system and back every claim above with tests, a verification notebook and measured results.

How each of these choices was made and why the alternatives were rejected, is set out in Section 4.

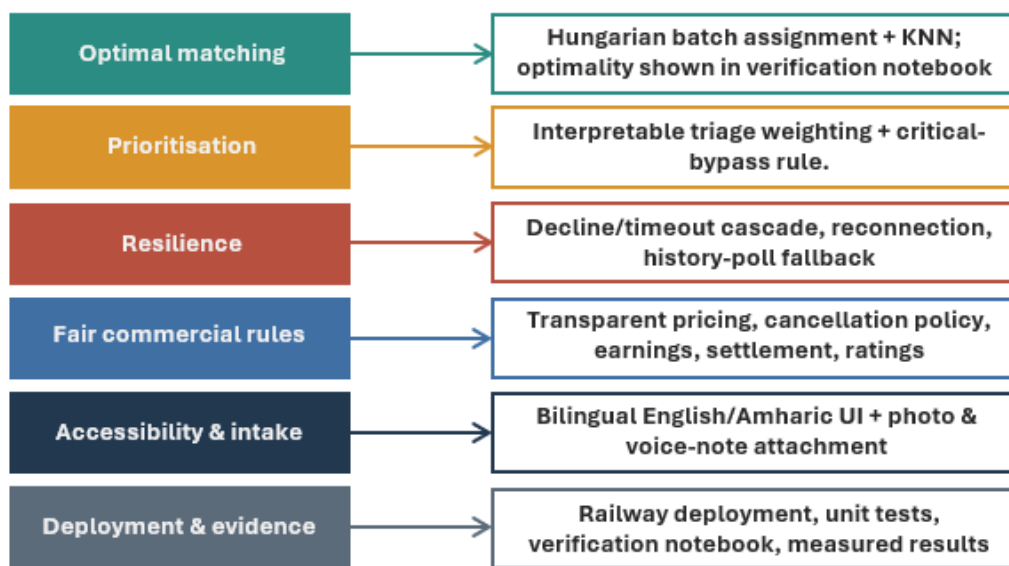


Figure 3 - Six objectives mapped to the delivered components of the platform that realise them.

2. Background and Related Work

2.1 The assignment problem

Matching operators to requests is an instance of the linear assignment problem: given the cost of pairing each operator with each request, find the one-to-one pairing that minimises the total cost. The Hungarian method of Kuhn (1955), refined by Munkres (1957) and treated comprehensively by Burkard, Dell’Amico and Martello (2009), solves this exactly and reliably fast. The time it takes grows manageably as more operators and requests are added, rather than exploding the way a brute-force check of every possible pairing would. The project uses the rectangular variant exposed by the SciPy `linear_sum_assignment` routine (Crouse, 2016; Virtanen et al., 2020), which handles unequal numbers of operators and requests and is efficient at the scale considered here.

The value of an exact solver in this setting is that the objective, the total travel distance across a set of simultaneous requests, is a global quantity that greedy per-request heuristics cannot be relied upon to optimise. A greedy nearest-operator rule assigns each request, in arrival order, to its own closest available operator. This is optimal for a single request but can be far from optimal for several because an early request may claim an operator that a later request needed more so the total depends on arrival order rather than geography. Figure 4 shows a small case in which greedy dispatch returns a total more than twice the optimum. Framing a batch of requests as an assignment problem and solving it exactly removes this order dependence and guarantees the minimum total for the batch, which is the property the project sets out to obtain and evidence. When only one request is pending, the problem collapses to selecting the single nearest capable operator, which is a one-nearest-neighbour decision in the sense of Cover and Hart (1967), and the system uses that lighter computation in the common single-request case.

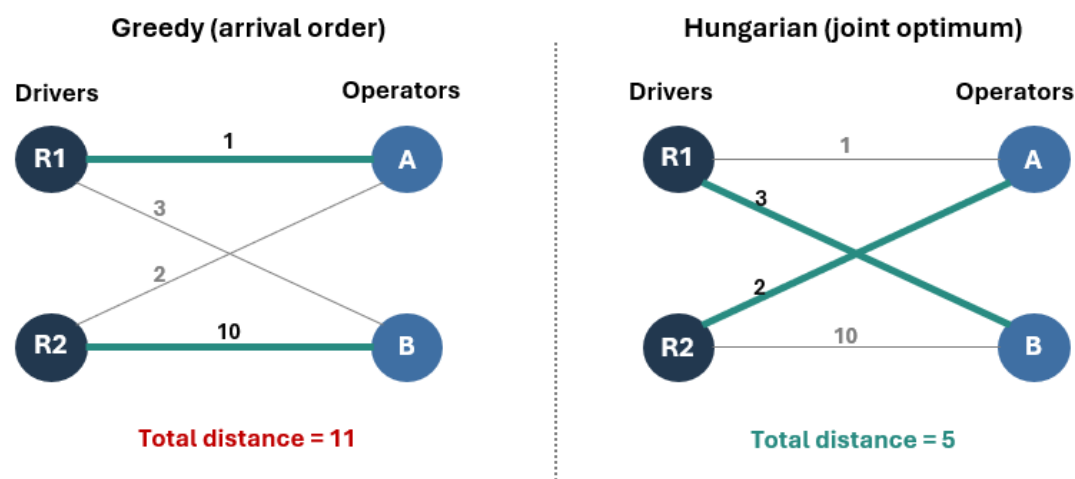


Figure 4 - Greedy versus optimal batch assignment; greedy follows arrival order and returns 11, while joint optimisation returns 5.

2.2 Dispatch in ride-hailing and logistics

Matching a pool of moving providers to incoming requests is central to ride-hailing and on-demand delivery, and it sits within the wider vehicle-routing and dynamic-matching literature surveyed by Toth and Vigo (2014). A recurring theme across that literature and in industrial practice is the move away from immediate greedy, nearest-driver dispatch towards batched matching, in which requests arriving within a short window are pooled and assigned jointly to reduce total waiting and travel. Xu et al. (2018) frame large-scale order dispatch on a commercial platform as a combined learning and planning problem rather than a greedy heuristic, while Alonso-Mora et al. (2017) show that high-capacity ride-sharing can be served by repeatedly solving a trip-to-vehicle assignment over short time windows. Reinforcement-learning methods for this type of matching have since become an active research strand (Qin, Zhu and Ye, 2022). RoadAid follows this general direction at a smaller scale, where the number of simultaneous requests is modest, and an exact solver is tractable. It also differs from a purely engineering treatment by measuring the benefit of the batched, optimal approach against the greedy alternative rather than assuming that the optimisation will improve the result.

The Ethiopian ride-hailing market gives these design decisions a concrete local grounding, alongside the academic literature reviewed above. RIDE, the market leader, which reports a pool of more than 90,000 drivers across Addis Ababa (RIDE, 2024), and ZayRide, a smaller but more geographically ambitious competitor founded in 2016 (Disrupt Africa, 2019), both show that GPS-based, app-mediated dispatch is already an accepted pattern among Ethiopian users; neither, however, offers towing or mechanical assistance, and neither prioritises requests by urgency or matches providers by anything beyond human-led allocation. ZayRide's extension of the same model into delivery and on-demand ambulance dispatch (Disrupt Africa, 2019) is a useful validation point, as it shows that the technical pattern this project uses, matching a mobile provider to a

distressed party by location and tracking the response in real time, is viable in this market beyond ordinary ride-hailing. The most direct comparator is Smart Auto Services PLC, an Addis Ababa roadside-assistance company offering towing, battery, fuel, tyre and lockout services to a paying membership through its own team of technicians (Smart Auto Services PLC, 2026). Its membership base is evidence that demand for the service exists, but its dispatch is manual and its capacity is capped by the size of its own team, since it has no open marketplace of independent operators. Smart Assist, a Tanzanian roadside-assistance service delivered through a dedicated app by Imatic Technologies (Smart Assist, 2025), is the closest African platform in overall design. Its existence supports the claim that an app-based roadside-assistance service is viable in a market with infrastructure constraints comparable to Ethiopia's. Overall, this local landscape shows real, proven demand for roadside assistance and a user base already comfortable with GPS-based apps. Still, no existing platform combines an open marketplace of independent operators with optimisation-based dispatch and severity-aware prioritisation, which is the gap this project addresses.

2.3 Prioritisation and multi-criteria weighting

Distinguishing urgent from routine requests is a prioritisation problem studied under triage in emergency medicine and more generally under multi-criteria decision analysis (Triantaphyllou, 2000). Structured, multi-level triage instruments such as the Emergency Severity Index (Gilboy et al., 2011) show that a small number of directly reported factors can be combined into a reliable and defensible acuity signal, and this project follows the same principle. Two modelling routes were available: a classifier learned from historical outcomes, or an interpretable rule that combines a few reported factors into a severity signal. The project adopts the latter, for two reasons. First, no historical dataset of request outcomes exists from which a classifier could be trained or validated so that a learned model would be unsupported by evidence. Second, for a safety-adjacent decision that affects who receives help first, an interpretable and auditable rule is more defensible than an opaque model and it is easier for operators and the platform owner to understand and trust. Following the practice of established breakdown services, the design asks the client directly for the relevant facts as structured inputs rather than inferring them, and it converts those facts into both a discrete tier and a continuous weight, as described in the methodology. The attendant risk that a user might exaggerate their situation to obtain faster service, is acknowledged and discussed later in the report.

2.4 Real-time delivery and the distance model

Two further technical choices underpin the platform. The first is the use of a persistent bidirectional channel for time-critical interactions, including offering a job, tracking an operator and propagating a status change, instead of repeated polling. Django Channels provides this through the WebSocket protocol (Fette and Melnikov, 2011),

routing messages to named groups so each is delivered only to the participants who need it. Polling, in which the client periodically asks the server for updates instead of receiving them as they occur, is retained as a fallback when the channel is unavailable. This introduces a short delay between a change occurring and the client detecting it, but avoids complete loss of state updates. The choice responds directly to the connectivity conditions of the target market, where mobile coverage is intermittent, and smartphones are comparatively scarce (Kemp, 2024; GSMA, 2024). The second choice is the use of great-circle or Haversine distance (Sinnott, 1984) as the cost in the assignment problem. Road distance would be more accurate, but it would require a routing-service call for every request-and-operator pair on every dispatch, which is costly and fragile on an intermittent connection. The computational cost of the two approaches is not comparable and this was measured rather than assumed: a single Haversine calculation takes under half a microsecond, so building the cost matrix for a generously oversized batch of fifty requests against fifty operators, 2,500 pairwise distances, takes roughly 1.2 milliseconds in total. A routing-service call, by contrast, is a network round trip rather than local arithmetic - typically tens to hundreds of milliseconds per request, rate-limited and metered per call, and requiring a separate call for every request-operator pair rather than one in-process loop. Straight-line distance is therefore not only simpler but measurably cheaper and remains adequate for choosing between candidate operators at city scale; a routing service is reserved for the driver-facing ETA, where a single call per completed job is affordable. This trade-off favours reliability and is consistent with the operating context.

3. Data, Resources and Project Inputs

3.1 Operational and derived data

Every assistance request is represented as a record that carries the pickup coordinates, for towing the drop-off coordinates as well, the service type of either towing or repair, the vehicle type, a free-text problem description and four severity inputs collected directly from the client, namely the number of passengers, whether children are on board, whether there is a medical concern and whether the location is hazardous. A request may also carry an optional photograph and a short voice note, which are forwarded to the operator as human context. Operators carry a live GPS position, a service type, an active or inactive status and a running average rating. These fields are the raw inputs to the dispatch decision.

From the raw inputs, the system derives, at the time of dispatch, a discrete priority tier of standard, elevated, or critical and a continuous priority score in the interval from zero to one. From the coordinates, it derives great-circle (Haversine) distances between each candidate operator and the request, and for towing, a fare from a fixed rate table. None of these derived values is stored as a primary input, each is recomputed from the current state so the decision is always based on live information. All state is persisted in PostgreSQL through the Django object-relational mapper, which provides the transactional guarantees the dispatch logic relies upon.

The persisted model is explicitly compact. Two core entities carry the system: an account record that represents both drivers and operators under a single user-type discriminator, and a job record that represents the full lifecycle of a request from submission to completion or cancellation. Figure 5 shows these two entities, their key fields, and the two relationships that connect them, namely that one driver submits many jobs and one operator is assigned many jobs. Table 1 sets out the full data inventory, distinguishing the raw inputs, the values derived at dispatch and the records retained afterwards.

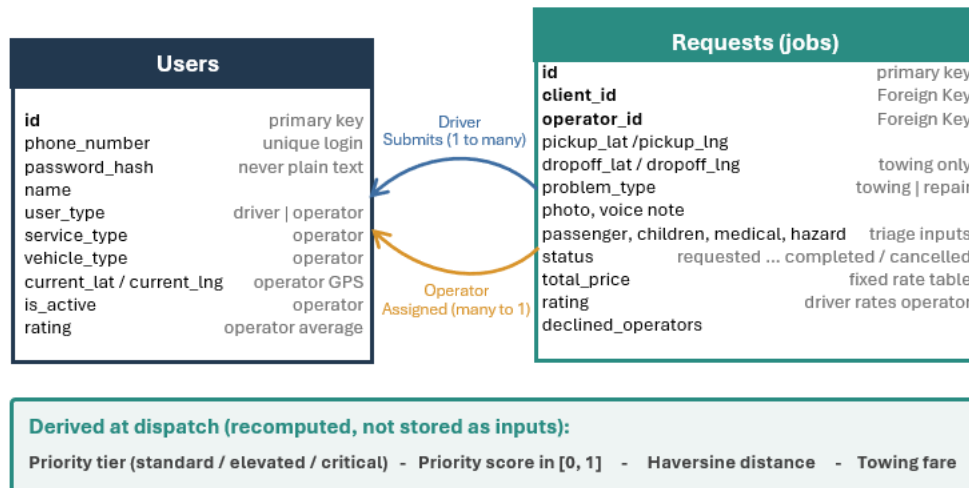


Figure 5 - Data model. A driver submits many jobs; an operator is assigned to many jobs over time. Green box: values computed at dispatch.

Table 1 - Data inventory: raw inputs, values derived at dispatch and records retained afterwards.

Item	Source	Role in the system
Pickup and drop-off coordinates	Client device GPS at request time	Locates the request; the drop-off sets the towing route and fare.
Service type (towing or repair)	Client selection	Filters the operator pool to those who can serve the request.
Vehicle type	Client selection	Matched against operator capability.
Problem description	Client free text	Human context for the operator.
Photo and voice note (optional)	Client capture	Extra human context, forwarded to the operator.
Severity inputs: passengers, children, medical, hazard	Client structured inputs	Drive the derived priority tier and score.
Operator position, service, status, rating	Operator app and history	The live candidate pool for matching.
Priority tier and score	Derived from severity inputs	Weight the assignment toward more urgent requests.
Haversine distance	Derived from coordinates	The cost in the assignment problem.
Fare	Derived from the fixed rate table and distance	Transparent towing price shown before service.
Job record and lifecycle timestamps	Persisted in PostgreSQL	Auditable history of who, what, price, and outcome.

Rating	Client after completion	Feeds the operator average and accountability.
Worked cases and synthetic conflict scenarios	Constructed for evaluation	Confirm optimality and the greedy comparison.

3.2 Evaluation data

The platform is newly built, so no historical log of real dispatch decisions exists against which matching quality could be measured. The engine is therefore evaluated on two forms of constructed data. The first is a set of hand-designed worked cases whose optimal assignments are known by exhaustive enumeration, used to confirm that the solver returns the true optimum. The second is a set of synthetically generated conflict scenarios, in which operator and request positions are drawn uniformly at random within a bounding box around Addis Ababa, roughly latitude 8.92° to 9.05° and longitude 38.70° to 38.83°, used to compare the optimal assignment against a greedy baseline across many independent trials. This is an appropriate and honest evaluation strategy given the data available. Its principal limitation is that synthetic uniform demand may not reflect real spatial demand patterns stated in Section 3 and revisited in Section 7.

3.3 Resources, tools, and assisted development

The backend is built on Django, with Django Channels providing WebSocket support and PostgreSQL providing persistence. The optimisation and numerical work uses SciPy, whose `linear_sum_assignment` routine solves the assignment problem (Virtanen et al., 2020), together with NumPy. The two mobile applications are written in Flutter and Dart, and the system is deployed on Railway. Supporting services include the Google Maps and Distance Matrix APIs for mapping and the driver-facing estimated time of arrival, and the record and audio players packages for capturing and playing back the voice note that accompanies a request. Both mobile applications were developed and tested primarily against the Android emulator given that Android accounts for over 93% of mobile operating system use in Ethiopia against under 7% for iOS (StatCounter, 2026), making an Android-first build the appropriate scope for the target user base. Table 2 summarises the stack and the responsibility of each part.

Table 2 - Technology stack and the responsibility of each part

Concern	Technology	Responsibility
Backend	Django with Django Channels	Business logic, request lifecycle, and WebSocket dispatch.
Real-time transport	WebSockets (Django Channels)	Job offers, live tracking and status updates to named groups.
Data store	PostgreSQL via the Django ORM	Single source of truth with transactional guarantees.
Optimisation and numerics	SciPy (linear_sum_assignment), NumPy	Exact assignment and supporting computation.
Mobile clients	Flutter and Dart	The driver app and the operator app.
Media capture and playback	Record, audio player packages	Capture and replay of the request voice note.
Mapping and ETA	Google Maps and Distance Matrix APIs	Driver-facing route and estimated time of arrival.
Deployment	Railway	Hosting reachable from any network.
Assisted development	Claude (Anthropic)	Accelerated implementation and documentation, reviewed and verified by the group.

3.4 The Depannini base

The platform was not written from an empty repository. It is a fork of the open-source Depannini codebase. Depannini supplied the plumbing of a two-sided service application: a Django backend with JSON Web Token authentication, hashed passwords and driver and operator registration, a relational schema for service-request and operator records, the basic job state machine of accept, decline and progression from accepted to ongoing to completed, a Channels-based WebSocket transport, and two Android Flutter applications. Much of this was backend-only or partly wired, as registration, rating and history had working endpoints but not fully developed.

The dispatch engine was built from the ground up: nearest-operator matching by K-Nearest-Neighbours, the Hungarian batch optimisation, the priority-weighted cost

matrix and critical-bypass rule and the decline-or-timeout cascade. Further modifications implemented were the pricing and cost estimate, the cancellation policy and fee model, operator earnings and platform-fee settlement, ratings and reviews, live tracking and assignment notifications, the bilingual English and Amharic interface, Ethiopian phone-number validation, the photo and voice-note intake, the SOS path, and the live deployment on Railway. Where the base offered a backend without a client, as with registration, rating and history, the group built the missing side.

Using a fork was a project decision. Starting from a running scaffold allowed the group to focus more of its effort on the optimisation, prioritisation and Ethiopian-context features required by the brief. The trade-off was inherited technical debt, most visibly the unresolved concurrency and load-performance questions carried into the limitations. Part of the early work therefore involved auditing the base to establish what could be retained, what required modification and what needed to be rebuilt.

3.5 Constraints and ethical considerations

Three considerations bear on the data handled by the platform. The first is data protection. The system stores personal and location data, including names, hashed passwords, phone numbers used as the login credential and precise GPS coordinates, so passwords are never held in plain text and access is limited to the parties to a job. The photograph and voice note attached to a request are personal data supplied by the client to obtain help and are shared only with the assigned operator. The second consideration is fairness. Since the priority signal is derived from self-reported severity inputs, a user could in principle exaggerate their situation to obtain faster service. The use of an interpretable and auditable rule, discussed in Section 2.3 and detailed in the methodology, partly addresses this risk although the residual exposure is acknowledged in Section 7. The third consideration is evaluation validity. The absence of historical demand data means that the engine is evaluated on synthetic scenarios. These are sufficient to establish optimality and support the greedy comparison but they cannot demonstrate performance under real demand, which is a limitation carried into the discussion.

4. Methodology and Approach

Verification was not left until the end. Development followed a V-model lifecycle (Forsberg and Mooz, 1991; Sommerville, 2016), shown in Figure 6, which pairs each design stage with a matching test stage: requirements against acceptance testing, system design against verification on the deployed platform and the detailed dispatch, pricing and cancellation logic against unit and integration testing backed by a dedicated verification notebook. Section 5 reports coverage against this plan.

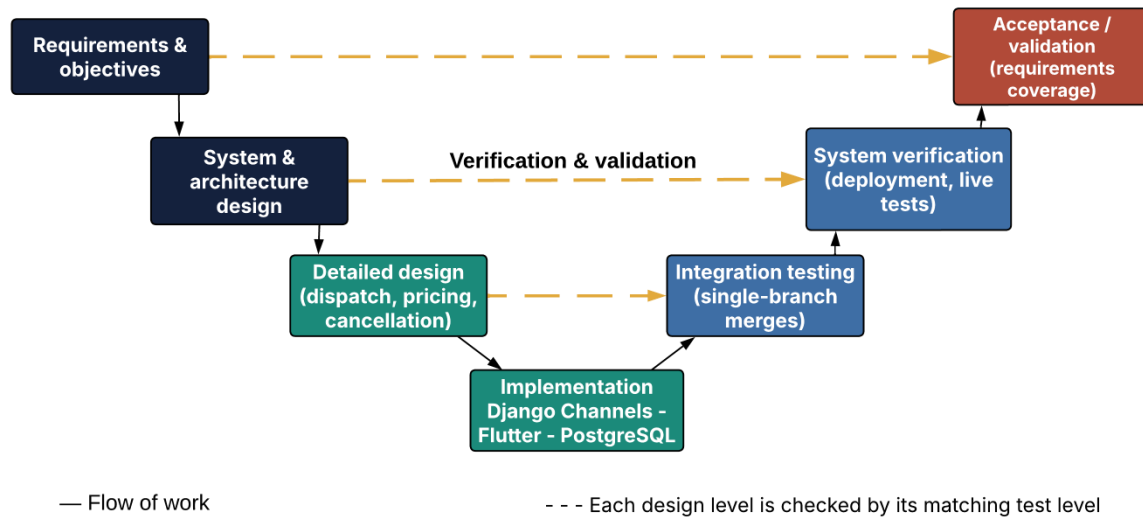


Figure 6 - V-model development lifecycle: each design level on the descending arm is checked by a matching test level on the ascending arm.

4.1 Dispatch as a two-mode optimisation

The dispatch decision is partitioned by the number of requests pending at the time of assignment, as shown in Figure 7. When only one request is pending, there is nothing to jointly optimise, and the system uses K-Nearest-Neighbours with $K = 1$: it selects the eligible operator, meaning one of the correct service type, currently active and with a stored GPS position, that minimises the Haversine distance to the pickup. For a single request, this is exactly optimal.

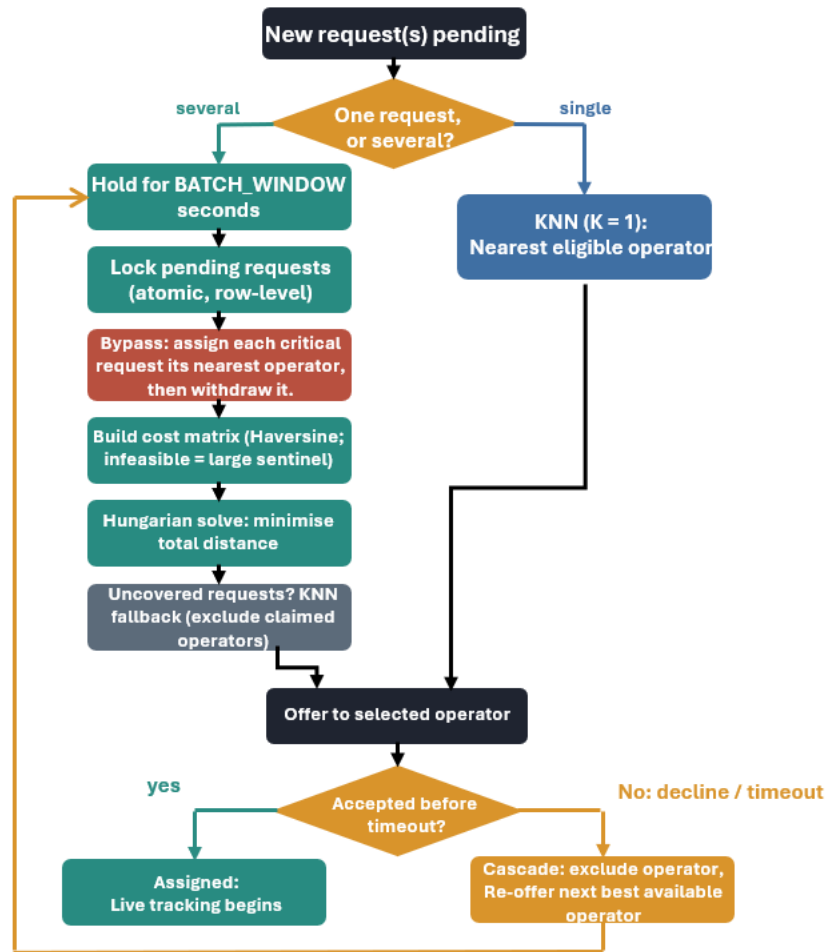


Figure 7 - Dispatch decision flow: a single request uses KNN and a batch is bypassed for critical cases, solved by the Hungarian algorithm, with a Hungarian fallback and a cascade.

When several requests may arrive close together, a freshly created request is held for a short batching window, a single named constant (`BATCH_WINDOW_SECONDS`, set to ten seconds), before assignment. During this window, the system locks the set of still-pending requests using a row-level database lock, which prevents two concurrent dispatch passes from assigning the same operator twice. It then constructs a cost matrix in which rows are requests, columns are eligible operators, and the entry is the Haversine distance from the request to the operator when the pair is feasible, or a large sentinel value when the operator cannot serve that service type or has no location on record. Solving this matrix with the SciPy `linear_sum_assignment` routine (Virtanen et al., 2020) returns the one-to-one assignment that minimises the total distance across the whole batch. Any request that the assignment cannot cover, for example when requests outnumber eligible operators, falls back to the Hungarian pool for reassignment, with operators already claimed in the same pass excluded so that the fallback cannot re-issue a taken operator.

Holding a request before dispatch introduces the platform's main design trade-off. This is a small instance of the online-matching problem: committing to an assignment

before the full sequence of requests is known can be worse than waiting for additional requests (Karp, Vazirani and Vazirani, 1990). The batching window therefore balances two competing quantities: the additional waiting time experienced by the request and the potential improvement in assignment quality. A zero-second window reproduces greedy, order-dependent dispatch, while a longer window allows more requests to be considered jointly and increases the chance of a better allocation at the cost of a longer wait. The ten-second setting is consequently a bounded compromise between these effects (Yan et al., 2020). Figure 8 works through a concrete batch of three requests and three operators and shows that the optimal assignment need not assign each request to its nearest operator. At the expected number of simultaneous requests, the solver's cubic cost is negligible. The selection of KNN for single requests and the Hungarian algorithm for batches was therefore based on the problem structure and the system's requirements.

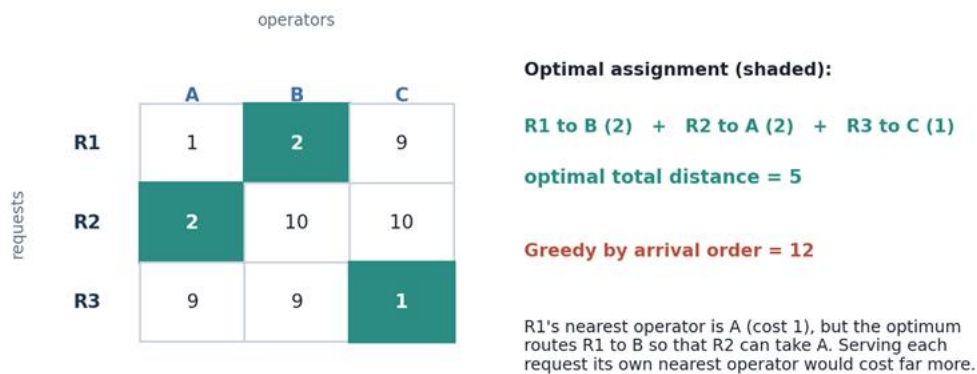


Figure 8 - A worked batch of three requests and three operators, where the Hungarian assignment minimises the total distance.

K-Nearest-Neighbours ($K = 1$) was selected for single-request dispatch because it is simple, fast, interpretable, and exactly optimal when only one request is pending. The Hungarian algorithm was selected for multi-request dispatch because it gives a proven guarantee of the minimum-total assignment across simultaneous requests, available as a single SciPy call. Three alternatives were considered and rejected. Reinforcement learning would need extensive historical training data that does not yet exist and would be harder to interpret for a safety-adjacent decision (Xu et al., 2018; Qin, Zhu and Ye, 2022), so it is recorded as future work rather than dismissed outright. Road routing via Dijkstra (1959) or A* (Hart, Nilsson and Raphael, 1968) was rejected for the matching decision itself, since it needs a full network graph. Road-network mapping is known to be substantially less complete in this region than in the well-mapped cities these algorithms are usually evaluated against (Barrington-Leigh and Millard-Ball, 2017), road accuracy is instead reserved for the driver-facing ETA where a single call per job is affordable. A random-assignment baseline was not implemented as it would only ever serve as a naive lower bound, unlike the greedy baseline used for comparison in Section 5.2.

4.2 Priority-weighted cost matrix

The four severity inputs are converted into a tier and a score in two complementary forms summarised in Figure 9. A rule function returns standard, elevated or critical from the reported factors: a medical concern is always treated as critical, a hazardous location combined with children on board or a lone driver is critical, a single risk factor is elevated, and the absence of risk factors is standard. A second function returns a continuous score in the interval from zero to one by combining the same factors additively and clamping the result. In the batch cost matrix, each request row is multiplied by a weight that increases with severity: $\text{weight} = 1 + \text{score} \times (\text{ceiling} - 1)$, where the ceiling is a single configurable constant (set to 6.0 in production, so a maximum-severity request's row is weighted six times its raw distance) and the score is the continuous value described above. A standard request with a score of zero keeps a weight of 1, leaving its row unchanged, while a higher score pushes the weight linearly up toward the ceiling.

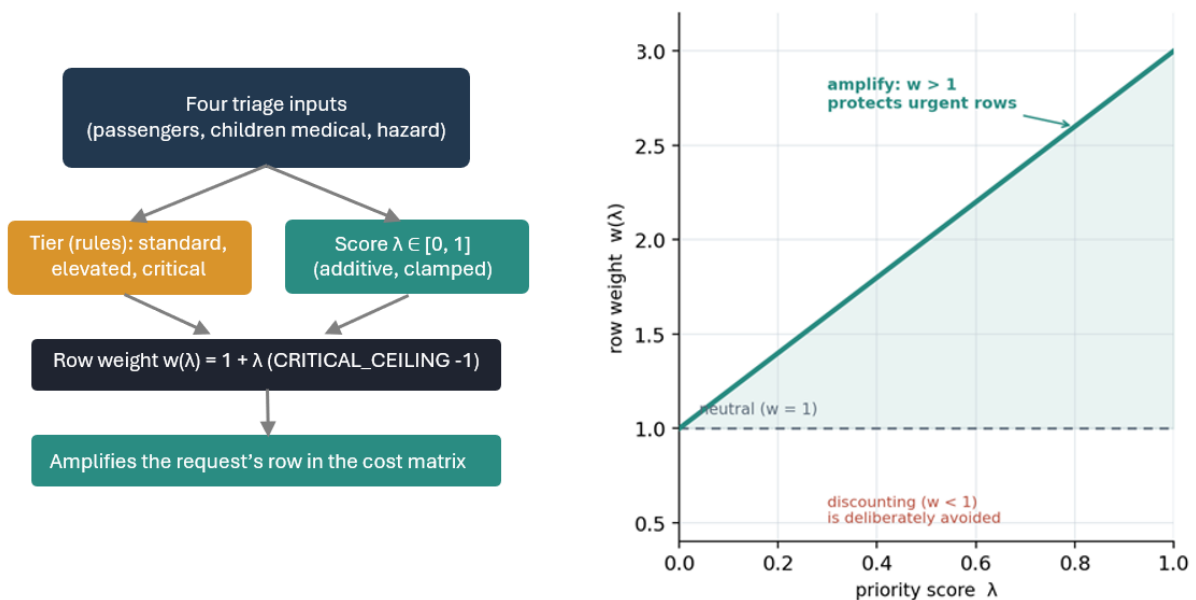


Figure 9 - The priority-weight model: triage inputs yield a tier and a score, interpolated into a multiplicative weight that amplifies the request's row.

The solver minimises the total, so shrinking a priority row makes that row matter less, and the routine request keeps the near operator. Raising the row does the opposite: a bad assignment on the urgent job costs more, so the solver keeps the near operator on that job. Table 12 is the check: unweighted and discounted both give the near operator to the routine request; only amplification gives it to the priority request. The continuous score serves the same purpose. Three named tiers cannot separate “medical only” from “medical + hazard + children”, both are critical but λ is 0.40 against 0.90 (Table 11).

4.3 Critical bypass

Weighting can only raise the odds that a severe request wins a contested operator, it cannot guarantee it because the outcome still depends on everything else in the batch. For the most severe cases, raised odds were not judged sufficient, so critical-tier requests are removed from the joint optimisation entirely. Before the Hungarian step runs, each critical request is assigned its true nearest operator by KNN and that operator is withdrawn from the pool offered to everyone else. This guarantee has a bounded, understood cost: removing an operator from a minimum-cost assignment problem can only leave the remaining problem's optimum the same or worse, never better, so the batch total for the non-critical requests can be marginally higher than under a single joint optimisation. That is an accepted trade-off, a small loss of group-level efficiency in exchange for certainty on the cases where a delayed response matters most.

4.4 Decline cascade, reconnection and fallback

The design assumes the network and the participants will not behave ideally. When an offered operator declines a request or does not respond within a timeout, that operator is recorded against the request and the request is released back into the pool of pending requests, where it competes for the next available operator alongside genuinely new requests through the same KNN-or-Hungarian logic described in Section 4.1, rather than being walked down a fixed ranked list on its own. This lets a request that has already lost time on a declined offer still benefit from joint optimisation, and it means a request never stalls silently. On the client side, WebSocket connections are kept alive with periodic application-level pings and are re-established after a drop, because the deployment platform terminates idle connections. As a further safeguard, both applications poll the job history periodically, so that a state change such as a cancellation is detected even when the live socket is temporarily unavailable.

4.5 Pricing and cancellation

Two commercial rules were modelled as pure framework-independent functions so that they could be unit-tested separately from the database and request cycle. Towing fares are calculated from a single fare table containing a base charge and a per-kilometre rate for each vehicle type. Pricing is therefore fixed and transparent rather than demand-sensitive. A demand-aware multiplier was considered but set aside for future work in Section 7 because there was insufficient project time to design, calibrate and test it safely, rather than because of a technical limitation.

The cancellation policy is the more developed of the two decision models and Figure 10 sets out its logic. A cancellation is free before an operator has accepted, free within a short grace window measured from the moment of acceptance and free whenever the operator cancels. The operator is excluded from re-dispatch of the same request, so

that an operator cancellation behaves as a late decline rather than a chargeable event. A driver cancellation after the grace window is charged the greater of a flat stage-dependent base amount (200 ETB once an operator is en route, 500 ETB once work is underway - Appendix D1) and a per-kilometre amount levied on the distance the operator actually travelled toward the client, at half the vehicle's fare rate (worked examples in Appendix D2). The result is capped at the estimated fare so that cancelling can never cost more than completing, giving the rule: $fee = \min(fare, \max(base, rate \times distance\ travelled))$.

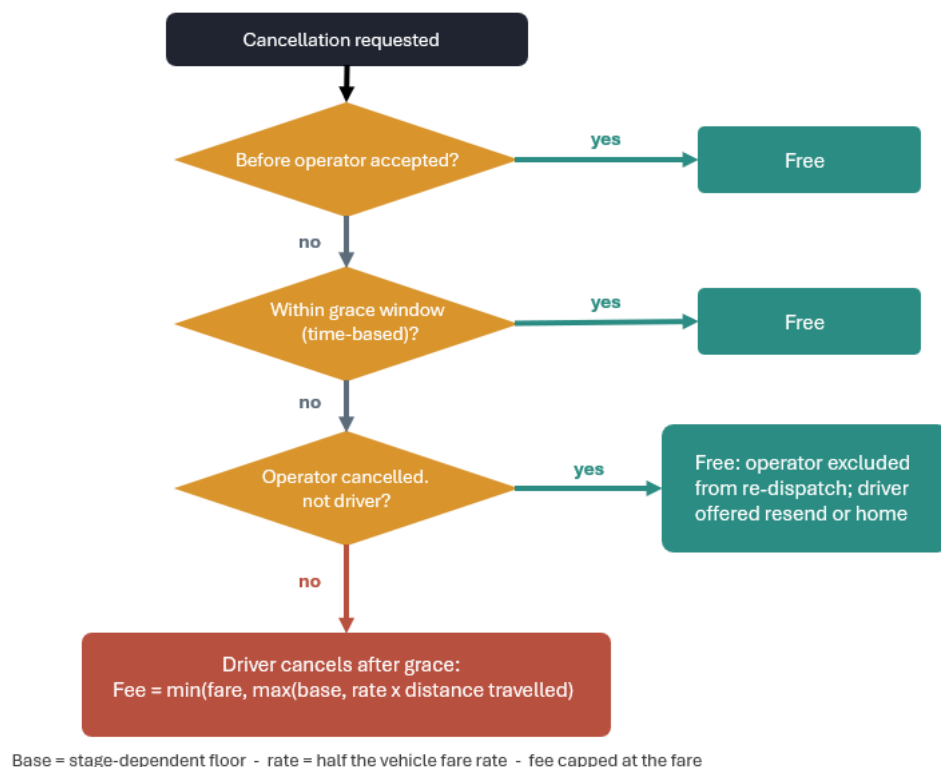


Figure 10 - The cancellation decision model: free before acceptance, within the grace window or on an operator cancellation and otherwise a capped distance-aware fee.

The grace window is measured in time rather than distance because a clock continues to advance when GPS updates are unavailable, allowing the rule to continue operating under poor connectivity. The flat base also provides a minimum defined charge after the grace window even when little or no distance has been recorded, preventing the fee from falling to zero in low-telemetry conditions. A worked example illustrates the intended behaviour. Example (Appendix D.2). An operator has driven 3 km of a 5 km approach. Cancellation rate is half the fare rate. If the fare rate is 50 ETB/km, the distance part is 75 ETB. That is compared with the stage base (200 ETB accepted, 500 ETB ongoing) and the larger amount is charged, never above the fare. If no GPS has arrived, the distance part is 0 and the base is charged. Operator cancellation is 0 ETB and the driver can resend a request to receive help or can abort trying. These rules are

intended to avoid arbitrary charges for a stranded driver and to maintain a predictable cancellation process.

4.6 Evaluation strategy

In the absence of a ground-truth dispatch log, the correctness and value of the engine are established through three forms of evidence. The first is proof-by-comparison: on small cases, the solver's output is compared with the true optimum obtained by brute-force enumeration. The second is a randomised simulation that compares the optimal assignment with a greedy baseline across many independent conflict scenarios, showing how often and by how much the optimisation improves on the heuristic and whether it ever produces a worse result. The third is a suite of automated unit tests at each layer, covering dispatch, priority weighting and cancellation. These tests establish the behaviour of individual functions and help guard against regressions. Together, the three forms of evidence provide a suitable evaluation basis for an optimisation and decision project of this type in the absence of a held-out operational dataset, and the results are reported in Section 5.

4.7 Transactional integrity and concurrency

Dispatch reads and writes shared rows under concurrency, so the batching pass runs inside a database transaction and takes a row-level lock on the pending requests. This stops two requests from ever being offered the same operator. The same principle, that the server is the single authority, applies to money: a price or cancellation fee is always recomputed on the server, and the driver is never trusted to supply a figure, so a compromised or defective client cannot alter a charge. A driver cancellation is told apart from an operator cancellation by a single attribute rather than a second status, which keeps the state machine small. The compact schema these guarantees rest on is the one shown in Figure 5. These are ordinary but essential guarantees for a system that coordinates real-world actions and payments.

4.8 Estimated time of arrival and mapping

Once an operator has been matched, the driver is shown the operator's live position and an estimated time of arrival. Whereas the assignment cost uses straight-line distance for the reasons given in Section 2.4, the estimated time of arrival is a driver-facing figure for a single, already-decided pairing, so a more accurate road-based estimate is warranted, and it is obtained from the Google Distance Matrix service. Separating the two uses of distance, a cheap and robust proxy for the many-to-many matching decision and an accurate road estimate for the one-to-one display, keeps the dispatch decision fast and resilient while still presenting the driver with a realistic expectation. It confines the dependency on an external routing service to a point in the flow where a single call per job is acceptable.

Table 3 - *The main methodological decisions with the reasoning and the trade-off accepted for each.*

Decision	Rationale	Trade-off accepted
Batching window before the Hungarian solver	Creates the opportunity for a globally optimal joint assignment	A few seconds of added latency for a better total.
Amplify, not discount, the priority row.	A minimising solver protects an amplified row and neglects a discounted one.	Slightly larger totals for non-urgent requests.
Critical bypass out of the joint solver	Guarantees the nearest operator for the most severe cases	A small, bounded loss of group-level efficiency.
Time-based grace window (not distance)	A clock advances even when GPS does not, so it degrades gracefully offline.	Ignores the distance actually travelled within the window.
Flat base as a floor on the cancellation fee	Guarantees a defined charge in low-telemetry conditions	A minimum charge even when little distance was travelled.
Haversine for matching, road distance for the ETA	Keeps the many-to-many decision fast and robust, with accuracy where it is shown	A straight-line cost is approximate for matching.
Server-side recomputation of every price and fee	The client can never alter a charge	An extra server computation per request.

5. Implementation, Results and Evaluation

5.1 System architecture and request lifecycle

The deployed system is organised in three tiers. A Django Channels backend serves conventional HTTP endpoints for authentication and record management, and maintains WebSocket channels for real-time offers, live location updates and status changes. PostgreSQL holds all persistent state, and two Flutter applications provide the driver and operator experiences. Real-time messages are routed through named channel groups, one per operator for incoming offers and one per active request for status and location, which keeps each message targeted to exactly the participants who need it.

A complete request proceeds through a fixed lifecycle. The driver submits a request, optionally attaching a photo and a voice note, and the backend creates the record and holds it in the batching window. The dispatch engine then selects the operator or operators and pushes the offer to the chosen operator's channel. On acceptance, the driver begins tracking the operator's live position. On completion, the driver pays and rates the operator, after which the job is recorded in both parties' history (see Appendix A for a full screenshot walkthrough of this journey). The architecture is the one shown in Figure 2, and the entire platform is deployed on Railway.

On the driver side, the operator's position is rendered on a live map with a polyline route that updates reactively as new location broadcasts arrive over the WebSocket channel, giving the driver continuous visual confirmation of the operator's approach rather than a static ETA figure alone. This required resolving a key mismatch between the location broadcast payload and the client's parser, which had silently prevented the driver's marker from updating. Once corrected, the same periodic-broadcast mechanism (Section 4.4) that drives resilience elsewhere in the platform keeps both the driver's map and the operator's own view synchronised.

5.2 Dispatch engine: results

The dispatch engine runs as part of the live backend rather than as a set of standalone scripts, and its behaviour is evidenced by the verification notebook `Verify_Dispatch.ipynb`, which executes cleanly from beginning to end (the full executed output is reproduced in Appendix B). Mode selection is correct: a single request is served by KNN and receives its true nearest operator, while several simultaneous requests are routed through the Hungarian solver. In a worked three-request scenario, the Hungarian assignment produces a total of 1.395 km, matching the brute-force global minimum of 1.395 km computed by exhaustive enumeration, and every operator is assigned to at most one request. On a constructed conflict designed to expose the weakness of greedy dispatch, the greedy total is 9.721 km against the Hungarian total of 9.412 km.

The comparative behaviour was then examined across 200 randomised conflict scenarios. The Hungarian assignment was strictly better than greedy in 96 of the 200 scenarios (48 per cent) and tied in the remaining 104 (52 per cent), which are the cases in which greedy already reached the optimum. The average improvement was 17.9 per cent in the scenarios where Hungarian was better, and it never produced a worse total. Figure 11 reproduces this comparison independently. In the scatter on the left, every point lies on or below the equal line, showing that the optimisation is never worse than greedy; the histogram on the right shows the distribution of improvement in the contested cases. Service-type feasibility is respected in every assignment, and the edge cases also behave safely. When no eligible operator exists, the engine returns an empty result without raising an exception. The engine excludes operators with no stored location. When requests outnumber eligible operators, uncovered requests fall back to the pool of requests to use the Hungarian dispatch method while excluding operators already claimed in the same pass, so no operator is offered twice.

The algorithm is exercised by the automated script `test_dispatch_3.py`, which runs to completion without error, and by a priority-tier integration test posted through the live view stack. The row-level lock used by the batching window is exercised directly by the dedicated concurrent-dispatch stress test (`test_concurrent_dispatch_stress.py`), run against a real PostgreSQL database provisioned through Docker: across 75 concurrent requests over five rounds, no operator is ever assigned to more than one request at the same time. The lock itself is implemented as a row-level `select_for_update()` inside an atomic transaction, as described in Section 4.7.

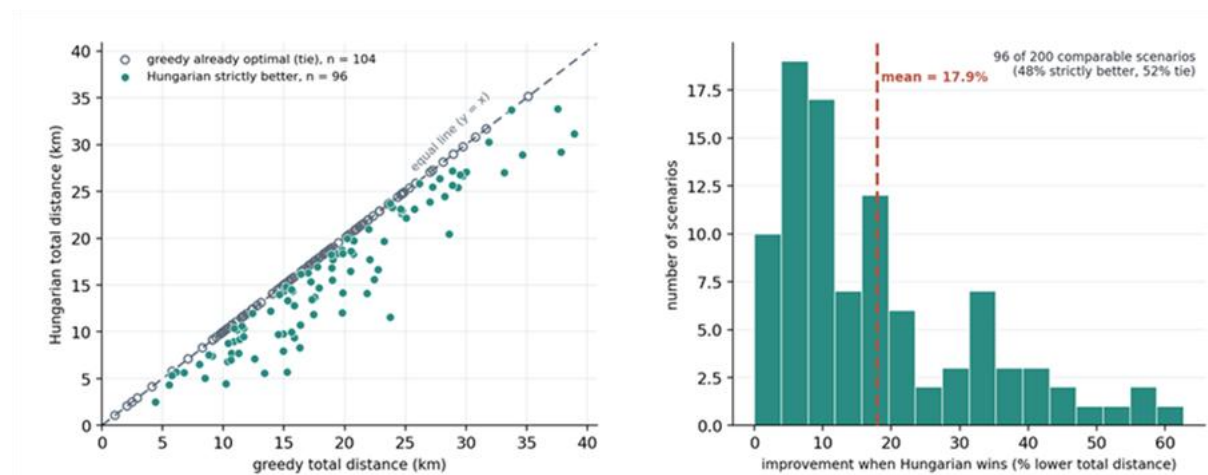


Figure 11 - Hungarian versus greedy dispatch across 200 random conflict scenarios; every point lies on or below the equal line, so the assignment is never worse than greedy.

The evaluation therefore supports a measured claim: the optimisation provides a material benefit in contested cases without producing a worse total in the tested scenarios. Its validity under real rather than modelled demand is considered in Section 5.7.

5.3 Priority model: results

Enumerating every combination of the four severity factors confirms that the priority model behaves as designed (full enumeration and test output in Appendix E; score definitions in Table 11, the amplify-versus-discount evidence in Table 12). A medical concern on its own is classified as critical; adding a hazardous location and children on board raises the continuous score above that of a single-factor case even though both are still labelled critical, which is the concrete evidence for why the continuous score was needed alongside the discrete tier. The amplify-not-discount decision was checked by direct computation on a deliberate tie: under a discounting weight the priority request is assigned the far operator; under an amplifying weight it is assigned the near operator, the opposite and intended result. This test also surfaced a second finding: the critical-bypass rule left part of the original weight range unreachable, since critical requests never reach the weighted step, so the CRITICAL_CEILING constant was recalibrated as a result. Table 13 summarises the automated test output that backs both findings.

5.4 Commercial and resilience features: results

The remaining features were implemented and verified against their intended behaviour. The cancellation policy behaves correctly across the full matrix of cases, confirmed both by live testing and by a dedicated suite of 17 unit tests (Appendix D): free before acceptance, free within the grace window, free and recorded when the operator cancels and charged on the distance actually travelled, subject to the guaranteed base and the fare cap when the driver cancels after the grace window. A driver whose operator cancels can choose to resend the request to another operator or return home, rather than being forced to abandon the request.

For payment and rating, the driver settles by cash, card, or mobile money and rates the operator on completion, and the operator sees completed jobs, total earnings, the fifteen per cent platform fee owed against the amount settled, and a reviews dashboard summarising ratings and per-job feedback (see Appendix C for the corresponding screens). For resilience, the cascade re-offers declined jobs automatically, client-side keepalive, reconnection and history-poll fallback keep both applications usable during the frequent host-lookup failures observed on the test network, a condition traced to the emulator and network environment rather than to the application code, and one the design explicitly anticipates. For media, a photo and a voice note attach to a request and are forwarded to the operator over the WebSocket. Both appear on the incoming-request and job-summary screens, giving the operator visual and spoken context before accepting.

Overall, these features provide a complete and auditable commercial model. Every completed or cancelled job records who was involved, what was charged, how it was

paid and how it was rated, and the operator's earnings view reconciles gross earnings against the platform's fifteen per cent commission.

5.5 Accessibility, intake and localisation: results

Both applications' interfaces are fully bilingual in English and Amharic. Ethiopian phone numbers, the +251 country code followed by nine digits beginning with 9 or 7, are validated on the client and enforced on the server by a regular-expression validator on the user model. A malformed number is therefore rejected at registration and in the administrative interface rather than stored. The localisation covers the complete interface of both applications, so a user who selects Amharic is not returned to English during the journey. This is relevant to a bilingual user base because switching language during a task would itself create a barrier. Accessibility is also supported by optional photo and voice-note attachments, which reduce the effort required to describe a problem for users who prefer not to type and provide the operator with the customer's own account before acceptance.

5.6 Requirements coverage

Measured against the platform's functional scope, all must-have functionality was delivered and deployed: registration and authentication, request submission with a price estimate, optimal matching, live tracking, completion, payment, rating and history, on both the driver and operator sides. The should-have features, meaning prioritisation, the cancellation policy, media attachments, and the bilingual interface, were delivered and evidenced. The could-have features, demand-based pricing and two-factor authentication were not implemented and are carried forward to future work in Section 7. The non-functional requirement of demonstrably optimal batch assignment was met and evidenced by the verification notebook. In contrast, the non-functional requirements on responsiveness under concurrent load were not yet measured and are carried into the limitations. Overall, the project delivered its core scope in full, and most of its extended scope and the principal shortfall is measured performance testing rather than missing functionality. Table 4 sets out the coverage in full.

Table 4 - Requirements coverage, with the priority, status and evidence for each.

Requirement	Priority	Status	Evidence
Register and authenticate	Must	Delivered	Live apps; phone-and-password authentication.
Submit a request with a price estimate	Must	Delivered	Fixed rate table; estimate shown before confirmation.
Optimal matching	Must	Delivered	KNN and Hungarian; verification notebook (Figure 11).
Live tracking	Must	Delivered	WebSocket location updates.
Complete, pay, rate and record history.	Must	Delivered	Full job lifecycle; earnings and reviews.
Prioritisation of urgent requests	Should	Delivered	Severity tier and score verified in Appendix E; priority-tier integration test.
Fair cancellation policy	Should	Delivered	17-test suite; model in Figure 10.
Photo and voice-note attachments	Should	Delivered	Captured and forwarded to the operator.
Bilingual English and Amharic interface	Should	Delivered	Full interface localisation.
Demand-based pricing	Could	Future work	Considered, not implemented (Section 7).
Two-factor authentication	Could	Future work	Not implemented (Section 7).
Real-time, demonstrably optimal dispatch	Non-functional	Met	Verification notebook; brute-force optimality.
Usable on an intermittent connection	Non-functional	Met	Cascade, reconnection, history-poll fallback.
Responsiveness under concurrent load	Non-functional	Not yet measured	Carried to limitations (Section 7).

Risk management was considered throughout development rather than only after problems arose - table 5 lists the main risks identified during development and the measures used to address them.

Table 5 - *Principal risks identified during development and how each was mitigated.*

Risk	Impact	Mitigation
Branch divergence from parallel work	Merges dropped code (a deleted view briefly broke the backend)	Single shared working branch; a session log per change; key backend files checked after every merge
No live payment gateway available	Cannot process real payments in the Ethiopian market	Payment and settlement mocked transparently within the app; a real gateway recorded as future work (Section 7)
Timeline pressure on the dispatch backlog	Several refinements could not all be finished before completion	Lower-priority items scheduled explicitly as future work rather than left unexplained
Priority weighting could protect the wrong request when only one operator was contested.	A critical request could lose its only available operator to a standard one.	Caught by the automated test suite before deployment; resolved by the critical-bypass rule (Section 4.3)
Manual deployment migrations	Deployed code briefly referenced new database columns before the migration that created them had run.	Incident resolved; adding a migration step to the release process recorded as future work (Section 7)

5.7 Threats to validity

The evaluation is a controlled simulation rather than a field study, so its limits should be stated plainly. The principal threat is external validity. The two hundred scenarios place operators and requests uniformly at random within a bounding box over Addis Ababa, whereas real demand is clustered in space and time, concentrated on arterial roads, the city centre, and peak hours. Uniform placement produces contention between requests at a rate that need not match reality, so the size of the improvement reported here, a mean of 17.9 per cent on contested batches, is evidence of the method's advantage under the modelled conditions and not a forecast of the field figure. The directional result, that the assignment is never worse than greedy, holds whatever the spatial distribution and only the frequency and scale of the gains would move under real demand.

Three further threats are controlled by design rather than left open. Greedy and Hungarian were run on identical scenarios under the same feasibility rules and distance metric, so the paired comparison isolates the assignment rule as the only difference, which is why it is appropriate. Distance is modelled as straight-line Haversine rather than road-network travel, an approximation both methods share, so the comparison remains fair even though absolute distances understate real journeys. The objective minimised is total batch distance, a system-level measure, so a single request can be assigned a slightly farther operator for the good of the batch, with the critical-bypass rule of Section 4.3 the deliberate exception that protects the most urgent cases. Responsiveness under concurrent load is not measured, so no claim is made about latency at scale.

6. Discussion

The evidence supports the project's central claim: framing simultaneous dispatch as an assignment problem (Kuhn, 1955) and solving it exactly produces measurably better matching than greedy dispatch, at the cost of a short, bounded batching delay. This benefit was supported by proof-by-comparison and a randomised simulation, rather than asserted. The distinction matters because of what the evidence does and does not establish. It establishes that, for the objective of minimising total travel distance over a batch, the optimisation is never worse than greedy and is frequently better on randomly generated geometry. It does not establish a specific improvement in real-world response time, which depends on the true spatial and temporal distribution of demand, and that distribution is not available. The reported figures should therefore be read as evidence of the method's correctness and of its advantage under the modelled conditions, not as a field measurement.

The prioritisation work illustrates the kind of analytical judgement the project sought to demonstrate in two respects. The first is the counter-intuitive finding that the priority weight must amplify rather than discount, which was tested against a concrete tie rather than accepted on first reasoning. The second is the recalibration prompted by the interaction between the priority weighting and the critical-bypass rule, which shows the reasoning about how two design decisions compose rather than treating them independently. The critical-bypass decision itself is a clear and defensible trade-off exchanging a small, bounded loss of group-level efficiency for certainty in the cases where the cost of delay is highest. The project also acknowledges an ethical dimension to prioritisation. Since severity is self-reported, a user could in principle exaggerate their circumstances to obtain faster service, an instance of the efficiency-versus-fairness tension studied in on-demand matching (Nanda et al., 2020). The design mitigates rather than eliminates this by keeping the rule interpretable and recording the reported factors against the request, so misuse is at least auditable. A fuller treatment would require verification or a reputation signal (Resnick et al., 2000) and is noted as future work.

The supporting decision models show that the same discipline was applied beyond the headline algorithm. Isolating the pricing and cancellation logic as pure functions made the behaviour easier to test and ensured that the backend remained authoritative: fees are recomputed by the server, and the client is not trusted to supply a charge. The cancellation rules also allow the system to continue operating when location data is incomplete. The framework on resilience, including cascades, reconnection and polling, is less visible than the optimisation itself, but it is necessary if a correct assignment is to remain usable in the target context.

Two broader observations follow. The first concerns generalisability. Although the platform was built for roadside assistance in Addis Ababa, the underlying decision,

assigning a pool of mobile providers to a stream of prioritised, geographically distributed requests in real time, recurs in ambulance dispatch (Bélanger, Ruiz and Soriano, 2019), on-demand courier and food delivery (Alonso-Mora et al., 2017), and field-service scheduling (Pillac et al., 2013), so the modelling approach and the evaluation method transfer to those settings with little change. The second concerns the value of designing for failure. Most of the engineering effort was directed not at just the optimisation itself, but at making that optimisation reliable and fair in an environment of dropped connections, declines and missing data. In an applied data-science project, the analytical idea is necessary but not sufficient. The surrounding decisions, about concurrency, graceful degradation and where authority sits, are what determine whether the idea survives contact with real use.

7. Limitations and Future Work

The project has several clear limitations. The main issues are described below together with the work that would be required to address them.

Synthetic evaluation only

The dispatch engine is shown to be optimal and never worse than greedy on constructed and randomly generated scenarios, but it has not been evaluated against real historical demand, which does not exist for this new service. Collecting real request logs and re-evaluating matching quality, wait times and the demand distribution against them is the most valuable next step, and would also open the way to learning-based dispatch that adapts to observed demand (Xu et al., 2018).

Unverified non-functional performance

Response time under concurrent load and the distribution of assignment times have not yet been measured under realistic concurrency, so the non-functional requirements on responsiveness remain unverified. This is distinct from the correctness of the concurrency lock itself, which is confirmed: run against a real PostgreSQL database, the dedicated stress test confirms that no operator is ever assigned to more than one request at the same time under concurrent load (Appendix E.5). What remains open is throughput and latency under load. Running the deployed platform under simulated concurrent traffic on PostgreSQL and reporting API latency and assignment time would close that remaining gap.

No live payment gateway

Payment and the platform fee are demonstrated within the application and settled manually, as no card gateway was available for integration in the target market. Integrating a real gateway or a mobile-money API, the dominant channel for digital payments across much of the region (Suri and Jack, 2016), is future work.

Placeholder constants

The grace-window length is seven seconds during testing and is intended to be increased to approximately ninety seconds in production. The cancellation-fee constants are also documented placeholders and should be calibrated against real cost data before the service is used commercially.

Demand-aware pricing and two-factor authentication

Two extended-scope features, a demand-sensitive pricing multiplier and two-factor authentication, were considered but not implemented within the timeline. Demand-sensitive pricing is well established as a mechanism for balancing supply and demand on on-demand platforms (Castillo, Knoepfle and Weyl, 2017). Both are self-contained

additions to existing surfaces, the pricing function and the authentication flow, and are recorded as future work.

Data protection and privacy

The platform handles personal and sensitive data, including phone numbers, live location, request photographs, voice notes and self-reported medical urgency. A full data-protection treatment was considered but could not be completed within the project timeline, and this remains a substantive limitation. A production deployment would require explicit and informed consent for the collection of location, images and audio; a defined retention and deletion policy; data minimisation in what each party can access; and compliance with the relevant framework, including Ethiopia's Personal Data Protection Proclamation No. 1321/2024 (Federal Democratic Republic of Ethiopia, 2024). Access control is the technical counterpart of this issue, so the two-factor authentication proposed above would also strengthen the authentication process. Further work on consent, retention and lawful processing would be required before real-world operation.

Operator verification and trust

A further enhancement is to add a layer of trust at operator registration through a government-issued certificate confirming that the operator is qualified and vetted, surfaced to drivers as a visible verified badge. This would let the platform guarantee that whoever provides a service has been checked, which directly addresses the accountability gap set out in Section 1.1.

8. Conclusion

RoadAid delivers a prototype roadside-assistance platform whose central decision, matching operators to requests, is made by an optimisation-based, priority-aware, and resilient dispatch engine, supported by transparent commercial rules and an accessible bilingual interface with a voice-note attachment. The project set out to answer three broad questions, and the evidence gathered across Sections 5 and 6 lets each be answered directly.

The first concerns the core matching decision: dispatch can be framed as an assignment problem and solved exactly, and doing so produces a better allocation than greedy dispatch at a bounded cost in latency. The Hungarian assignment was never worse than greedy across two hundred randomised batches and reduced total travel by a mean of 17.9 per cent on the contested cases. This answers the question in the affirmative, while the magnitude should be treated as a modelled result rather than a field measurement. The second concerns fairness under urgency: a severity signal drawn from self-reported inputs can be incorporated into the assignment while remaining interpretable and auditable. This is supported by the enumerated priority model, the amplify-not-discount test and the critical-bypass rule. The third concerns viability in the target setting. The system was made usable through connection recovery, decline handling, transparent pricing and a defined cancellation policy, together with a fully bilingual interface, low-effort request intake and live deployment. Performance under concurrent load remains unmeasured, and a full data-protection treatment is still outstanding.

Two lessons carry beyond this particular platform. The first is methodological: the value of the work lies less in the optimisation itself than in the discipline of evaluating and justifying each decision rather than asserting it. This is so that joint optimisation is demonstrated rather than assumed, prioritisation decisions are tested rather than trusted, and trade-offs are stated openly. The second lesson is that making an analytical idea usable takes more engineering than the idea itself, most of it spent on failure, concurrency, and fairness. The project has real limits, mainly its reliance on synthetic evaluation, its unmeasured behaviour under load, and its deferred privacy work. Within those limits, it stands as a coherent piece of applied data-science decision support built for its context and honest about its detail.

References

- Alonso-Mora, J., Samaranayake, S., Wallar, A., Frazzoli, E. and Rus, D. (2017) 'On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment', *Proceedings of the National Academy of Sciences*, 114(3), pp. 462-467.
- Barrington-Leigh, C. and Millard-Ball, A. (2017) 'The world's user-generated road map is more than 80% complete', *PLOS ONE*, 12(8), e0180698.
- Bélanger, V., Ruiz, A. and Soriano, P. (2019) 'Recent optimisation models and trends in location, relocation, and dispatching of emergency medical vehicles', *European Journal of Operational Research*, 272(1), pp. 1-23.
- Burkard, R.E., Dell'Amico, M. and Martello, S. (2009) *Assignment Problems*. Philadelphia: Society for Industrial and Applied Mathematics.
- Castillo, J.C., Knoepfle, D. and Weyl, E.G. (2017) 'Surge pricing solves the wild goose chase', *Proceedings of the 2017 ACM Conference on Economics and Computation*. Cambridge, MA: ACM, pp. 241-242.
- Cover, T.M. and Hart, P.E. (1967) 'Nearest neighbor pattern classification', *IEEE Transactions on Information Theory*, 13(1), pp. 21-27.
- Crouse, D.F. (2016) 'On implementing 2D rectangular assignment algorithms', *IEEE Transactions on Aerospace and Electronic Systems*, 52(4), pp. 1679-1696.
- Depannini Team (2025) Depannini: Vehicle Recovery & Roadside Assistance Platform [source code]. GitHub. Available at: <https://github.com/aamxh/depannini> (Accessed: 15 June 2026).
- Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, 1(1), pp. 269-271.
- Disrupt Africa (2019) 'Ethiopian startup behind ZayRide taxi app secures seed funding', *Disrupt Africa*, 17 January. Available at: <https://disruptafrica.com/2019/01/17/ethiopian-startup-behind-zayride-taxi-app-secures-seed-funding/> (Accessed: 28 August 2026).
- Federal Democratic Republic of Ethiopia (2024) *Personal Data Protection Proclamation No. 1321/2024*. Addis Ababa: Federal Negarit Gazette.
- Fette, I. and Melnikov, A. (2011) *The WebSocket Protocol. RFC 6455*. Internet Engineering Task Force (IETF). Available at: <https://www.rfc-editor.org/rfc/rfc6455> (Accessed: 17 August 2026).

Forsberg, K. and Mooz, H. (1991) 'The relationship of system engineering to the project cycle', Proceedings of the First Annual Symposium of the National Council on Systems Engineering (NCOSE), Chattanooga, TN, pp. 57-65.

Gilboy, N., Tanabe, P., Travers, D. and Rosenau, A.M. (2011) Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4. Rockville, MD: Agency for Healthcare Research and Quality.

GSMA (2024) Driving Digital Transformation of the Economy in Ethiopia. London: GSMA.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, 4(2), pp. 100-107.

Karp, R.M., Vazirani, U.V. and Vazirani, V.V. (1990) 'An optimal algorithm for on-line bipartite matching', Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing (STOC). Baltimore, MD: ACM, pp. 352-358.

Kemp, S. (2024) Digital 2024: Ethiopia. DataReportal. Available at: <https://datareportal.com/reports/digital-2024-ethiopia> (Accessed: 17 August 2026).

Kuhn, H.W. (1955) 'The Hungarian method for the assignment problem', Naval Research Logistics Quarterly, 2(1-2), pp. 83-97.

Munkres, J. (1957) 'Algorithms for the assignment and transportation problems', Journal of the Society for Industrial and Applied Mathematics, 5(1), pp. 32-38.

Nanda, V., Xu, P., Sankararaman, K.A., Dickerson, J. and Srinivasan, A. (2020) 'Balancing the tradeoff between profit and fairness in rideshare platforms during high-demand hours', Proceedings of the AAAI Conference on Artificial Intelligence, 34(2), pp. 2210-2217.

Pillac, V., Gendreau, M., Guéret, C. and Medaglia, A.L. (2013) 'A review of dynamic vehicle routing problems', European Journal of Operational Research, 225(1), pp. 1-11.

Qin, Z., Zhu, H. and Ye, J. (2022) 'Reinforcement learning for ridesharing: an extended survey', Transportation Research Part C: Emerging Technologies, 144, 103852.

Resnick, P., Kuwabara, K., Zeckhauser, R. and Friedman, E. (2000) 'Reputation systems', Communications of the ACM, 43(12), pp. 45-48.

RIDE (2024) RIDE 8294: Ethiopia's pride. Available at: <https://ride8294.com/>.

Sinnott, R.W. (1984) 'Virtues of the Haversine', Sky and Telescope, 68(2), p. 159.

Smart Assist (2025) Smart Assist [Mobile app]. Google Play. Available at: https://play.google.com/store/apps/details?id=com.itl.apps.smart_assist.

Smart Auto Services PLC (2026) Services. Available at:
<https://smartautoet.com/services/>.

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

StatCounter (2026) Mobile operating system market share Ethiopia. Available at:
<https://gs.statcounter.com/os-market-share/mobile/ethiopia>.

Suri, T. and Jack, W. (2016) 'The long-run poverty and gender impacts of mobile money', Science, 354(6317), pp. 1288-1292.

Toth, P. and Vigo, D. (eds.) (2014) Vehicle Routing: Problems, Methods, and Applications. 2nd edn. Philadelphia: Society for Industrial and Applied Mathematics.

Triantaphyllou, E. (2000) Multi-Criteria Decision Making Methods: A Comparative Study. Dordrecht: Kluwer Academic Publishers.

Virtanen, P. et al. (2020) 'SciPy 1.0: fundamental algorithms for scientific computing in Python', Nature Methods, 17, pp. 261-272.

World Health Organization (2018) Global Status Report on Road Safety 2018. Geneva: World Health Organization.

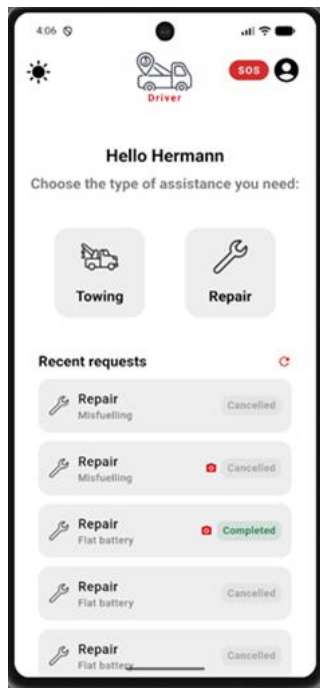
Xu, Z., Li, Z., Guan, Q., Zhang, D., Li, Q., Nan, J., Liu, C., Bian, W. and Ye, J. (2018) 'Large-scale order dispatch in on-demand ride-hailing platforms: a learning and planning approach', Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. London: ACM, pp. 905-913.

Yan, C., Zhu, H., Korolko, N. and Woodard, D. (2020) 'Dynamic pricing and matching in ride-hailing platforms', Naval Research Logistics, 67(8), pp. 705-724.

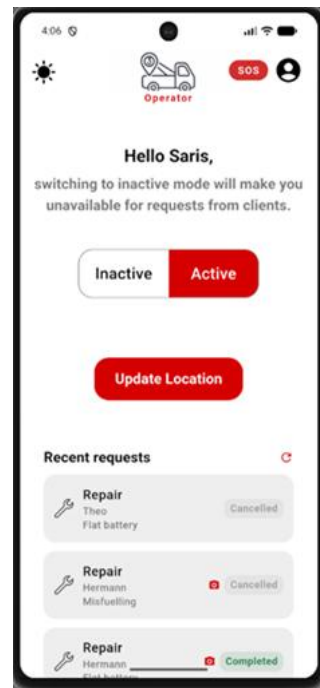
Appendices

Appendix A - End-to-End Job Walkthrough (Screenshots)

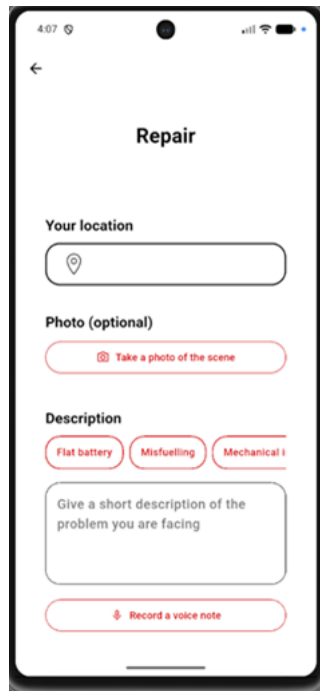
The following sequence of screenshots, captured from both the driver app and the operator app against the live deployment, demonstrates a complete repair job end-to-end: request submission, dispatch, navigation, arrival, pricing, payment, rating and job history on both sides. This evidence supports the end-to-end journey referenced throughout Section 5.



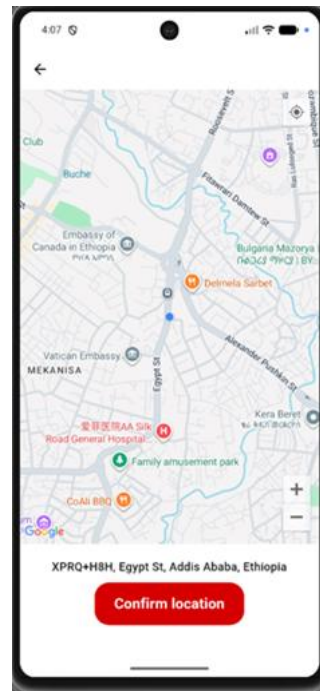
A.1. Driver - home screen: choose Towing or Repair, with recent requests



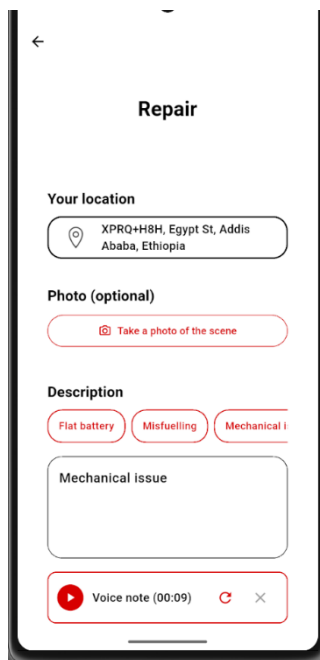
A.2. Operator - home screen: Active / Inactive toggle and recent requests



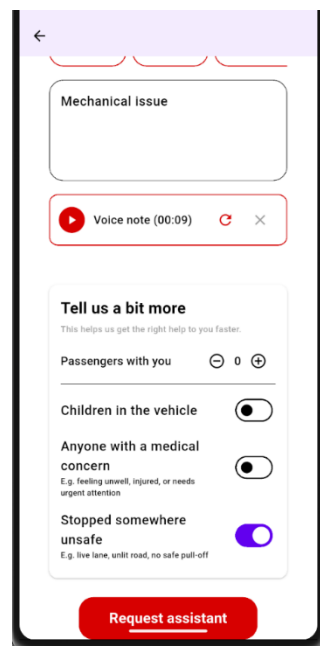
A.3. Driver - opens a Repair request: location, optional photo, description, voice note



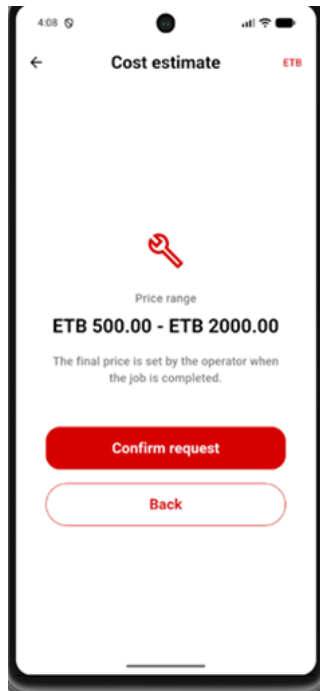
A.4. Driver - confirms the pickup location on the map



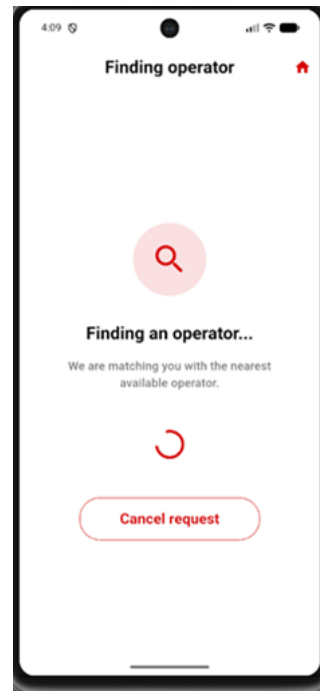
A.5. Driver - attaches a photo and enters a description ("Mechanical issue") with a voice note



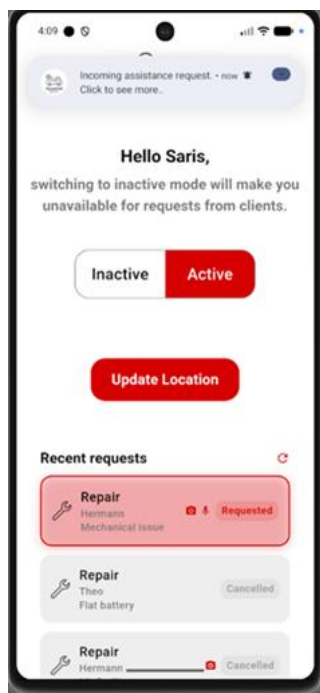
A.6. Driver - severity fields (passengers, children onboard, medical concern, hazardous location)



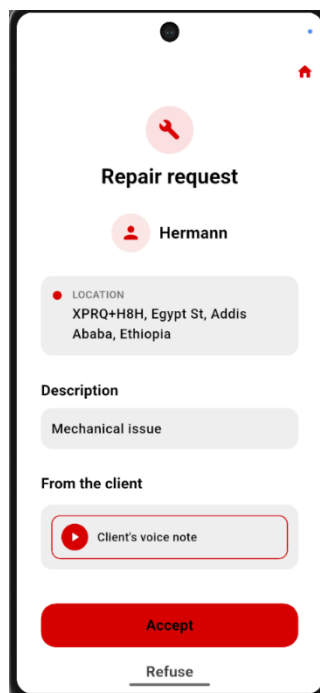
A.7. Driver - cost estimate (price range) before confirming the request



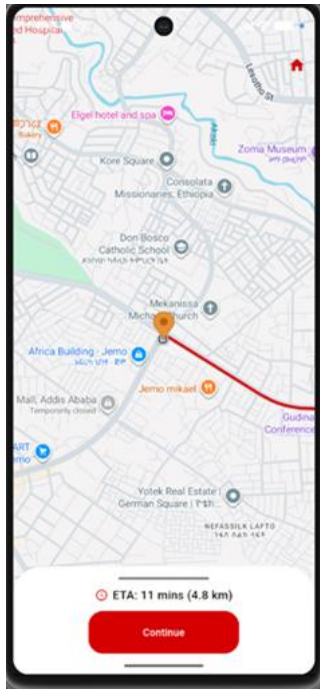
A.8. Driver - "Finding an operator..."



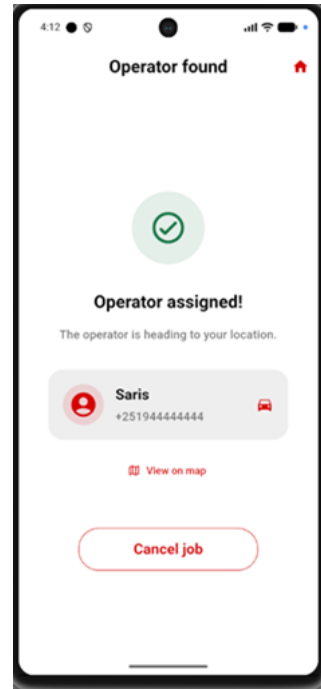
A.9. Operator - receives the incoming assistance request notification



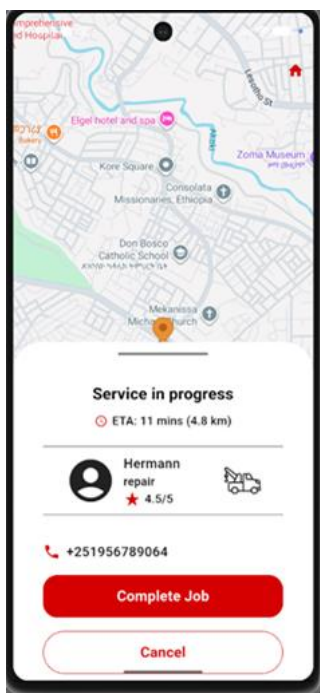
A.10. Operator - views the request details, photo, and voice note and accepts it



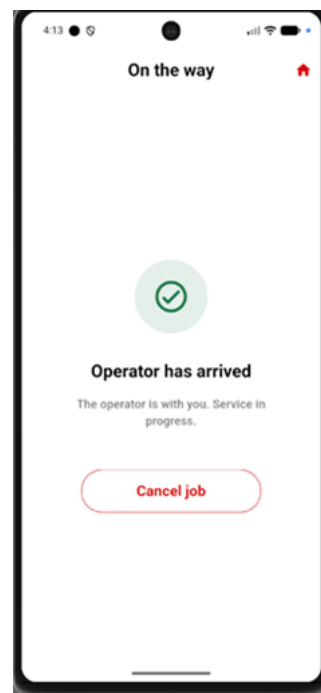
A.11. Operator - navigates toward the driver's location (ETA and route)



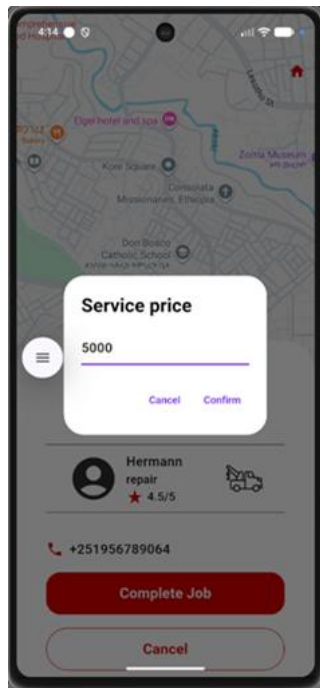
A.12. Driver - "Operator assigned!" with operator name and contact



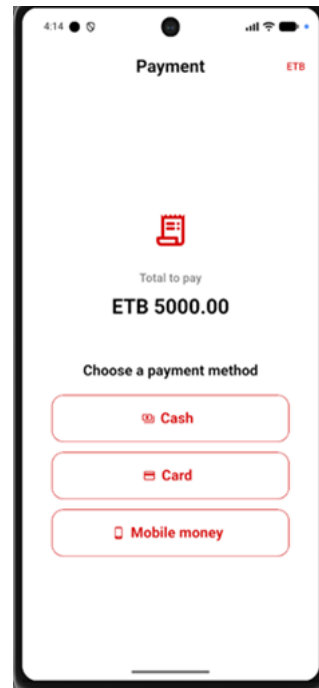
A.13. Operator - service in progress, with Complete Job available



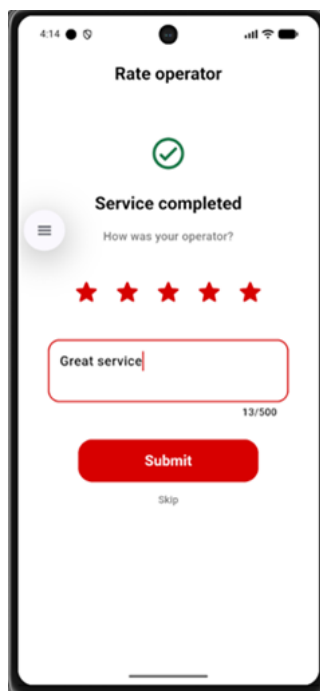
A.14. Driver - "Operator has arrived. Service in progress."



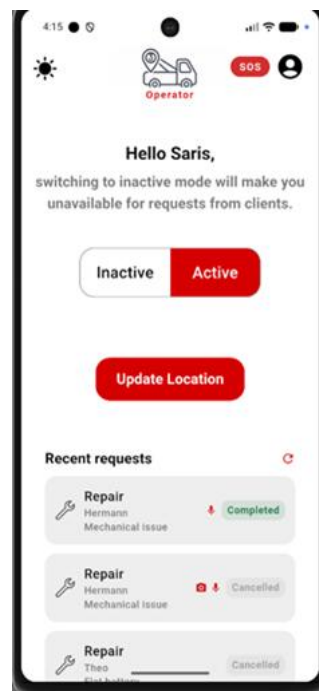
A.15. Operator - enters the service price to complete the job



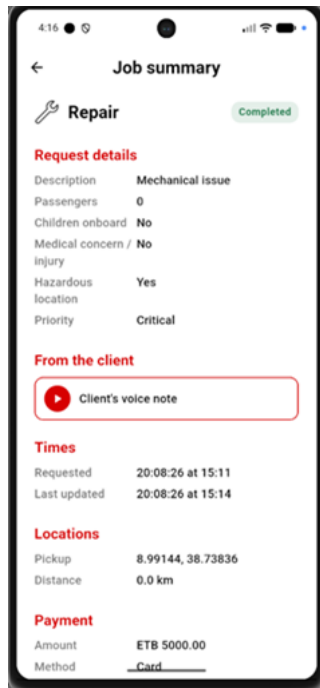
A.16. Driver - chooses a payment method



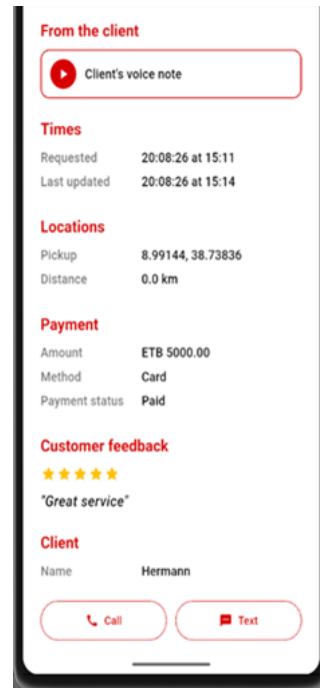
A.17. Driver - rates the operator on completion



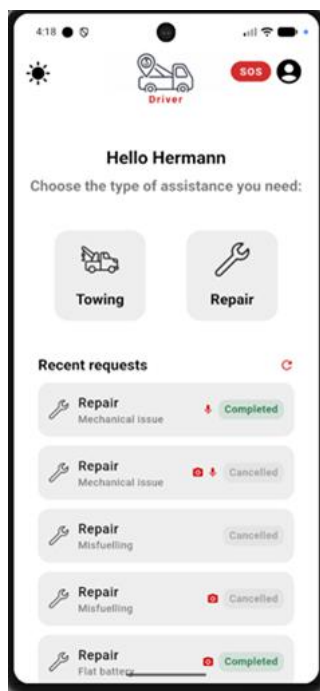
A.18. Operator - home screen shows the job as Completed



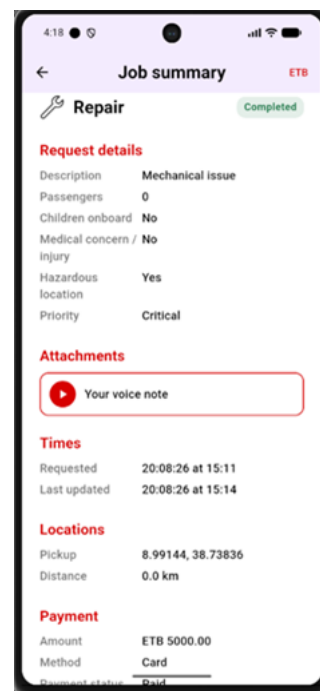
A.19. Operator - job summary: request details and the client's voice note



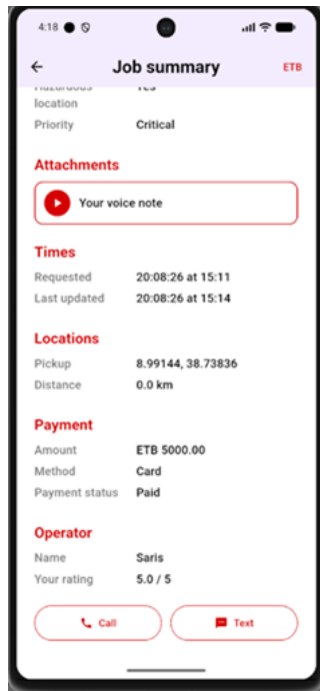
A.20. Operator - job summary continued: payment, customer feedback and client contact



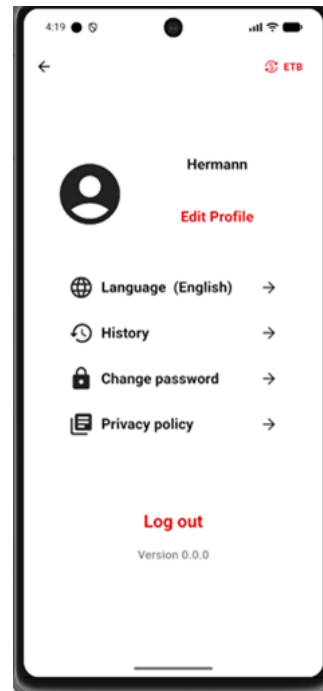
A.21. Driver - home screen shows the job as Completed



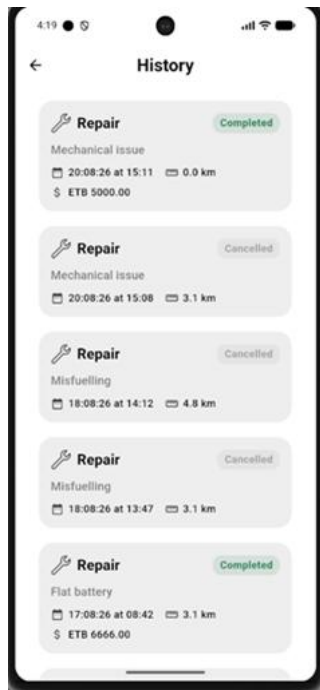
A.22. Driver - job summary: request details and attachments



A.23. Driver - job summary continued: payment and the operator's rating



A.24. Driver - profile menu, including access to History



A.25. Driver - history lists past requests and outcomes

Appendix B - Dispatch Verification

This appendix presents the executed evidence supporting the dispatch-engine claims made in Sections 4.1 and 5.2: that Mode 1 (K-nearest neighbour, $K = 1$) is selected for a

single pending request, that Mode 2 (the Hungarian / Kuhn-Munkres algorithm) is selected for multiple simultaneous requests, that the Hungarian assignment is mathematically optimal rather than merely better than one greedy example, and that service-type filtering and edge cases are handled without failure.

All output below was captured from a fresh, unmodified execution of the verification notebook (backend/depannini/core/tests/Verify_Dispatch.ipynb) and the automated script (backend/depannini/core/tests/test_dispatch_3.py), both run against the current dispatch.py implementation. No output has been edited by hand; console formatting, including the two typographical slips (“Cleint” / “Clent”) in the script's own print statements, is reproduced verbatim as it was produced.

Table 6- Appendix Table 1 - Summary of the evidence presented in this appendix. Full detail, including the raw output each row is drawn from, follows in B.1-B.7.

#	Check	Result
B.1	KNN selects the nearest eligible operator (brute-force cross-check)	PASS - Yonas at 0.598 km
B.2	Hungarian assignment matches the brute-force global optimum	PASS - 1.395 km = 1.395 km
B.3	Hungarian beats greedy on a contested-operator scenario	PASS - 9.412 km vs 9.721 km
B.4	Service-type filtering respected under Hungarian	PASS - every match correct.
B.5	200-trial comparison: Hungarian never loses to greedy	PASS - 48% strictly better, 52% tied, 0% worse
B.6	Edge cases (no eligible operator, missing GPS) handled without exception	PASS - no exception raised
B.7	test_dispatch_3.py automated script	Completed, exit code 0

B.1 Mode selection and independent verification (KNN)

With a single pending request, dispatch() selects Mode 1 (KNN, K = 1). The chosen operator was independently cross-checked by a brute-force scan ranking every eligible operator by distance:

Mode selected: KNN

<Request 101 (towing)> -> <Operator 1 Yonas - Bole (towing)> | distance = 0.598 km

All eligible (towing) operators ranked by distance:

Yonas - Bole 0.598 km

Dawit - Megenagna 2.338 km

Hana - Gerji 2.962 km

Selam - Piassa 6.035 km

PASS: KNN correctly selected Yonas - Bole at 0.598 km

Verify_Dispatch.ipynb, cells 3-4 - KNN mode selection and brute-force cross-check.

B.2 Hungarian assignment and brute-force optimality proof

With three simultaneous requests, dispatch() selects Mode 2 (Hungarian). The Hungarian total distance was then checked against a brute-force search over every feasible assignment, not just a single greedy baseline:

Mode selected: Hungarian

<Request 201 (towing)> -> <Operator 1 Yonas - Bole (towing)> | distance = 0.598 km

<Request 202 (towing)> -> <Operator 3 Dawit - Megenagna (towing)> | distance = 0.248 km

<Request 203 (repair)> -> <Operator 4 Betelhem - CMC (repair)> | distance = 0.549 km

Total distance across all assignments (Hungarian): 1.395 km

PASS: every operator in this batch was assigned to at most one request.

Hungarian total distance: 1.394904 km

Brute-force minimum (exhaustive search over every feasible assignment): 1.394904 km

PASS: Hungarian's result is mathematically proven optimal for this scenario - not just better than one greedy baseline, but equal to the best of every possible assignment.

Verify_Dispatch.ipynb, cells 4-4 (continued) - Hungarian assignment and exhaustive optimality proof.

B.3 Where greedy fails, and Hungarian does not

This scenario is constructed explicitly: two requests that are both individually closest to the same operator, to make the case for joint optimisation explicit rather than subtle:

Hungarian assignment:

<Request 301 (towing)> -> <Operator 11 Yonas - Far Bole (towing)> | 9.255 km
<Request 302 (towing)> -> <Operator 10 Dawit - Megenagna (towing)> | 0.156 km
Total: 9.412 km

Greedy assignment:

<Request 301 (towing)> -> <Operator 10 Dawit - Megenagna (towing)> | 0.156 km
<Request 302 (towing)> -> <Operator 11 Yonas - Far Bole (towing)> | 9.565 km
Total: 9.721 km

PASS: Hungarian total (9.412 km) <= Greedy total (9.721 km)

PASS: Hungarian total also matches the brute-force global optimum (9.412 km).

Verify_Dispatch.ipynb, cell 5 - Hungarian vs greedy on a contested-operator scenario.

B.4 Service-type filtering

A repair request must never be assigned a towing-only operator, even when Hungarian mode is active:

<Request 401 (repair)> -> <Operator 4 Betelhem - CMC (repair)> (repair) | 5.089 km
<Request 402 (towing)> -> <Operator 3 Dawit - Megenagna (towing)> (towing) | 0.896 km

PASS: every assignment matches the requested service type.

Verify_Dispatch.ipynb, cell 6 - service-type filtering under Hungarian mode.

B.5 200-trial randomised comparison

Sections B.2-B.3 demonstrate optimality on scenarios placed by hand, which is useful but open to the objection that the examples were cherry-picked. To address this, 200 randomly generated operator/request configurations were compared (seed = 42, fixed for reproducibility):

Trials run: 200
Comparable trials: 200
Hungarian strictly better in: 96 (48.0%)
Ties (greedy already optimal): 104 (52.0%)
Average improvement when Hungarian wins: 17.92% lower total distance

PASS: across 200 random configurations, Hungarian never loses to greedy KNN, and wins outright in 48% of comparable cases - the advantage from Section 5 is not cherry-picked.

Verify_Dispatch.ipynb, cell 7 - 200-trial randomised Hungarian vs greedy comparison.

B.6 Edge cases

Two situations the live dispatch path must handle without raising an exception: no eligible operator (wrong service type or nonactive), and an operator with no stored GPS location:

```
PASS (KNN): no eligible operator -> (None, None), no exception raised.  
PASS (Hungarian): no eligible operator -> [], no exception raised.  
PASS: an operator with no stored GPS location is correctly excluded from eligibility.
```

Verify_Dispatch.ipynb, cell 9 - edge-case handling.

B.7 Automated test-suite output (test_dispatch_3.py)

In addition to the notebook, the automated script `test_dispatch_3.py` (backend/depannini/core/tests/test_dispatch_3.py) exercises the same `dispatch.py` functions directly against a 13-operator mock fleet covering haversine distance, KNN mode for both service types, the 'no operator available' edge case, and Hungarian assignment for batches of 3 and 8 simultaneous requests. Full console output is reproduced below, unedited:

```
=====
TEST 1: Haversine distances from client to each operator
=====

Cleint: Meskel Square
Ali    (towing, active=True) -> 1.82 km
Kebede (repair, active=True) -> 2.98 km
Sara   (repair, active=True) -> 1.82 km
Yonas  (towing, active=True) -> 3.71 km
Meron  (repair, active=False) -> 6.15 km
Jack   (towing, active=True) -> 1.82 km
Reece  (towing, active=True) -> 2.98 km
Jaune  (repair, active=True) -> 1.82 km
Rosey  (repair, active=True) -> 442.85 km
Tigist (repair, active=True) -> 11.15 km
Dawit  (towing, active=True) -> 6.39 km
Hana   (repair, active=False) -> 6.00 km
Akaki  (towing, active=True) -> 14.95 km
-----

Cleint: Bole Airport
Ali    (towing, active=True) -> 6.89 km
Kebede (repair, active=True) -> 5.86 km
```

Sara (repair, active=True) -> 6.82 km
Yonas (towing, active=True) -> 8.68 km
Meron (repair, active=False) -> 10.92 km
Jack (towing, active=True) -> 6.89 km
Reece (towing, active=True) -> 5.86 km
Jaune (repair, active=True) -> 6.82 km
Rosey (repair, active=True) -> 447.96 km
Tigist (repair, active=True) -> 8.14 km
Dawit (towing, active=True) -> 10.29 km
Hana (repair, active=False) -> 8.73 km
Akaki (towing, active=True) -> 13.27 km

Cleint: Piazza

Ali (towing, active=True) -> 1.18 km
Kebede (repair, active=True) -> 1.63 km
Sara (repair, active=True) -> 1.02 km
Yonas (towing, active=True) -> 4.30 km
Meron (repair, active=False) -> 6.67 km
Jack (towing, active=True) -> 1.18 km
Reece (towing, active=True) -> 1.63 km
Jaune (repair, active=True) -> 1.02 km
Rosey (repair, active=True) -> 442.24 km
Tigist (repair, active=True) -> 10.81 km
Dawit (towing, active=True) -> 7.62 km
Hana (repair, active=False) -> 7.79 km
Akaki (towing, active=True) -> 17.12 km

Cleint: Kazanchis

Ali (towing, active=True) -> 2.59 km
Kebede (repair, active=True) -> 2.25 km
Sara (repair, active=True) -> 2.51 km
Yonas (towing, active=True) -> 5.01 km
Meron (repair, active=False) -> 7.46 km
Jack (towing, active=True) -> 2.59 km
Reece (towing, active=True) -> 2.25 km
Jaune (repair, active=True) -> 2.51 km
Rosey (repair, active=True) -> 444.00 km
Tigist (repair, active=True) -> 9.83 km
Dawit (towing, active=True) -> 7.68 km

Hana (repair, active=False) -> 7.11 km
Akaki (towing, active=True) -> 15.30 km

=====

TEST 2: Mode 1 - KNN nearest operator for TOWING request

=====

Client: Meskel Square (9.0192, 38.7525)

Assigned: Ali (ID 1) at 1.82 km

Client: Bole Airport (9.009, 38.799)

Assigned: Reece (ID 7) at 5.86 km

Client: Piazza (9.038, 38.747)

Assigned: Ali (ID 1) at 1.18 km

Client: Kazanchis (9.025, 38.763)

Assigned: Reece (ID 7) at 2.25 km

=====

TEST 3: Mode 1 - KNN nearest operator for REPAIR request

=====

Client: Meskel Square (9.0192, 38.7525)

Assigned: Sara (ID 3) at 1.82 km

Client: Bole Airport (9.009, 38.799)

Assigned: Kebede (ID 2) at 5.86 km

Client: Piazza (9.038, 38.747)

Assigned: Sara (ID 3) at 1.02 km

Client: Kazanchis (9.025, 38.763)

Assigned: Kebede (ID 2) at 2.25 km

=====

TEST 4: Mode 1 - no operator available (all inactive)

=====

Result: None - Expected: None

Result: None - Expected: None

Result: None - Expected: None

Result: None - Expected: None

=====

TEST 5: Mode 2 - Hungarian Algorithm for 3 simultaneous requests

=====

3 assignment(s) made:

Request 101 -> Ali at 1.82 km

Request 102 -> Reece at 1.24 km

Request 103 -> Sara at 2.63 km

=====

TEST 5: Mode 4 - Hungarian Algorithm for 8 simultaneous requests

=====

8 assignment(s) made:

Request 101 -> Yonas at 1.74 km

Request 102 -> Reece at 1.24 km

Request 103 -> Sara at 2.63 km

Request 104 -> Ali at 1.82 km

Request 105 -> Jack at 2.80 km

Request 106 -> Jaune at 1.82 km

Request 107 -> Kebede at 7.53 km

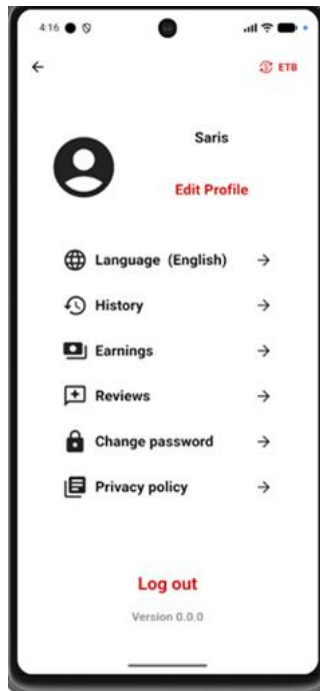
Request 108 -> Tigist at 8.14 km

All tests complete.

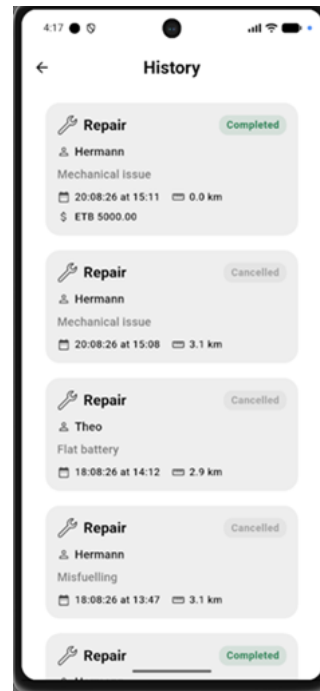
test_dispatch_3.py - full console output, exit code 0.

Appendix C - Operator Earnings, Fee Settlement and Reviews

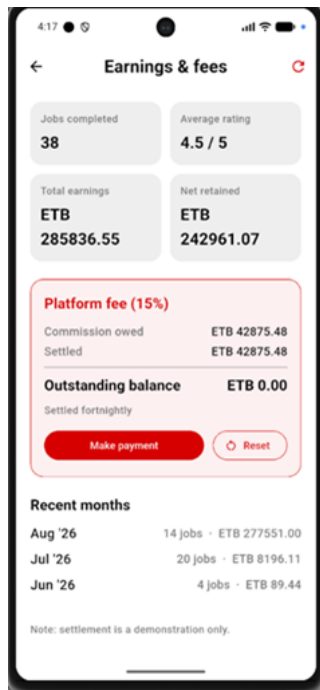
The following screenshots, captured from the operator app on 20 August 2026, evidence the commercial model described in Section 5.4: the operator's own job history, the earnings and platform-fee settlement view (gross earnings, the 15% commission owed and settled, and the fortnightly cadence), and the aggregate and per-job reviews the operator receives from clients.



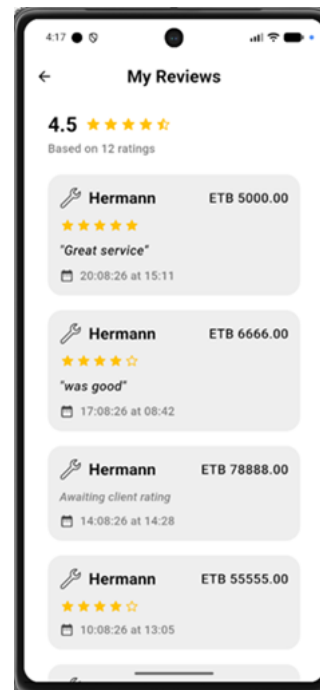
C.1. Operator - profile menu: Earnings and Reviews entries



C.2. Operator - history of past and current jobs



C.3. Operator - Earnings & fees: gross earnings, the 15% platform fee and fortnightly settlement



C.4. Operator - My Reviews: aggregate rating and per-job customer feedback

Appendix D - Cancellation Policy and Fee Model

This appendix presents the executed evidence supporting the cancellation-policy claims made in Sections 4.5 and 5.4: that an operator cancellation is always free to the driver, that a driver cancellation is free before acceptance and inside a grace window, that a cancellation after the grace window always carries a defined flat charge, that a genuinely long approach drive is compensated on top of that flat charge, and that the fee can never exceed the estimated fare. The policy is implemented as a single pure function, `compute_cancellation_fee()` in `backend/depannini/core/utils/cancellation.py`, so it can be exercised directly with plain numbers, independent of Django or the database.

D.1 Rate table

Table 7 - Appendix Table 2 - The constants below are the values `compute_cancellation_fee()` and `flat_base_for_stage()` are built from, taken directly from `core/utils/cancellation.py`.

Parameter	Value	Notes
Grace window	7 s (test) / ~90 s (recommended for production)	Starts at operator acceptance, not request time; time-based on purpose, so it still applies when GPS updates do not arrive.
Cancellation rate fraction	0.5 × the vehicle's fare per-km rate	The operator drove toward the client but did not complete the tow/repair, so the cancellation rate sits below the full fare rate.
Flat base - accepted	200 ETB	Minimum charge once the grace window has passed while the job was in the 'accepted' stage.
Flat base - ongoing	500 ETB	Minimum charge once the grace window has passed while the job was 'ongoing' (operator arrived / work started).
Fee formula	$\text{fee} = \min(\text{fare_cap}, \max(\text{flat_base}, \text{rate_per_km} \times \text{travelled_km}))$	$\text{travelled_km} = \max(0, \text{initial_distance_km} - \text{remaining_km})$. fare_cap is None for jobs unpriced until completion (e.g. repair).
Operator-initiated cancellation	0 ETB, unconditionally	Free to the driver at every stage; the operator instead accrues <code>cancellations_count</code> and is

		excluded from re-dispatch (view-layer logic, not unit-tested here).
--	--	---

D.2 Worked fee examples

Nine scenarios computed directly by calling `compute_cancellation_fee()` with the rate table above (`rate_per_km` = 25.0 ETB). They mirror the boundary cases exercised by the automated tests in D.3, expressed as concrete fee outcomes:

Table 8 - Appendix Table 3 - Computed live from `core/utlis/cancellation.py`. Examples 5-9 correspond to the `PerKmUpliftTests` and `FeeCapTests` assertions in D.3.

#	Scenario	Stage	Since accept	Distance driven	Fare cap	Fee (ETB)	Basis
1	Client cancels before an operator accepts	requested	n/a	n/a	none	0.00	pre_acceptance
2	Client cancels 5 s after acceptance (inside 7 s grace)	accepted	5 s	2.0 → 1.8 km	none	0.00	grace
3	Client cancels 60 s in; operator hasn't moved	accepted	60 s	2.0 → 2.0 km	none	200.00	flat_base
4	Client cancels after a short 2 km approach drive	accepted	60 s	5.0 → 3.0 km	none	200.00	flat_base
5	Client cancels after a long 10 km approach drive	accepted	60 s	12.0 → 2.0 km	none	250.00	per_km
6	Client cancels mid-job after a 24 km approach drive	ongoing	600 s	24.0 → 0.0 km	none	600.00	per_km
7	Long drive, but fee capped at a 100 ETB fare	accepted	60 s	10.0 → 0.0 km	100	100.00	per_km (capped)
8	Stationary, but a tiny 80 ETB fare caps even the base	accepted	60 s	2.0 → 2.0 km	80	80.00	flat_base (capped)

9	Operator cancels after driving the whole way	ongoing	600 s	12.0 → 0.0 km	none	0.00	operator_free
---	--	---------	-------	---------------	------	------	---------------

D.3 Automated test-suite output (test_cancellation.py)

test_cancellation.py (backend/depannini/core/tests/test_cancellation.py) is a pure unittest suite with no Django and no database, exercising every branch of the policy directly: operator cancellation (always free), pre-acceptance and in-grace cancellation (free), the flat stage base once the grace window has passed, the per-km uplift when it exceeds that base, and the fare cap. Run verbosely from backend/depannini with “python -m unit test core.tests.test_cancellation -v”, all 17 tests pass:

Table 9 - Appendix Table 4 - Summary of test_cancellation.py's five test classes.

Test class	What it checks	Tests	Result
OperatorCancellationTests	Operator cancellation is always free, at any stage	3	3/3 PASS
FreeCancellationTests	Pre-acceptance and inside-grace cancellation is free	3	3/3 PASS
FlatBaseTests	The flat stage base applies once the grace window has passed	6	6/6 PASS
PerKmUpliftTests	The per-km uplift applies once it exceeds the flat base	2	2/2 PASS
FeeCapTests	The fee never exceeds the estimated fare	3	3/3 PASS

Full verbose output follows below.

```
test_base_is_capped_at_a_small_fare_too
(core.tests.test_cancellation.FeeCapTests.test_base_is_capped_at_a_small_fare_t
oo) ... ok
test_fee_is_capped_at_the_estimated_fare
(core.tests.test_cancellation.FeeCapTests.test_fee_is_capped_at_the_estimated_fa
re) ... ok
test_no_cap_when_fare_is_unknown
(core.tests.test_cancellation.FeeCapTests.test_no_cap_when_fare_is_unknown) ...
ok
test_flat_base_helper_matches_stage
(core.tests.test_cancellation.FlatBaseTests.test_flat_base_helper_matches_stage) .
```

```
.. ok
  test_just_outside_grace_is_charged_the_base
(core.tests.test_cancellation.FlatBaseTests.test_just_outside_grace_is_charged_th
e_base) ... ok
  test_no_distance_data_still_charges_base
(core.tests.test_cancellation.FlatBaseTests.test_no_distance_data_still_charges_b
ase) ... ok
  test_ongoing_uses_the_higher_ongoing_base
(core.tests.test_cancellation.FlatBaseTests.test_ongoing_uses_the_higher_ongoing
_base) ... ok
  test_short_drive_below_base_still_charges_base
(core.tests.test_cancellation.FlatBaseTests.test_short_drive_below_base_still_cha
rges_base) ... ok
  test_stationary_operator_still_charged_the_base
(core.tests.test_cancellation.FlatBaseTests.test_stationary_operator_still_charged_
the_base) ... ok
  test_client_cancel_before_acceptance_is_free
(core.tests.test_cancellation.FreeCancellationTests.test_client_cancel_before_acc
eptance_is_free) ... ok
  test_exactly_at_grace_boundary_is_free
(core.tests.test_cancellation.FreeCancellationTests.test_exactly_at_grace_boundar
y_is_free) ... ok
  test_inside_grace_is_free
(core.tests.test_cancellation.FreeCancellationTests.test_inside_grace_is_free) ... ok
  test_operator_cancel_free_even_after_driving_the_whole_way
(core.tests.test_cancellation.OperatorCancellationTests.test_operator_cancel_free
_even_after_driving_the_whole_way) ... ok
  test_operator_cancel_while_accepted_is_free
(core.tests.test_cancellation.OperatorCancellationTests.test_operator_cancel_while
_accepted_is_free) ... ok
  test_operator_cancel_while_ongoing_is_free
(core.tests.test_cancellation.OperatorCancellationTests.test_operator_cancel_while
_ongoing_is_free) ... ok
  test_long_drive_lifts_above_the_base
(core.tests.test_cancellation.PerKmUpliftTests.test_long_drive_lifts_above_the_bas
e) ... ok
  test_uplift_must_clear_the_ongoing_base_too
(core.tests.test_cancellation.PerKmUpliftTests.test_uplift_must_clear_the_ongoing
_base_too) ... ok
```

```
Ran 17 tests in 0.001s
```

```
OK
```

test_cancellation.py - full verbose output, 17/17 passed.

D.4 Development record

The agreed design policy rationale is recorded directly in the module's own docstring `core/utils/cancellation.py` and explains the reasoning behind it - for example, why the grace window is time-based rather than GPS-based, and why the flat base guarantees a defined charge even when no telemetry is available. That docstring is reproduced in full at the top of `core/utils/cancellation.py` and is the authoritative record of the decision for this feature.

Appendix E - Priority Model and Resilience Evidence

This appendix presents the executed evidence supporting the priority-model claims made in Section 5.3 (that the continuous severity score distinguishes requests within the same discrete tier, and that amplifying rather than discounting a priority row is what protects its assignment) and the resilience claims touched on in Section 5.4, including the concurrency-locking guarantee verified against PostgreSQL in E.5.

E.1 Priority-weight model

Two functions run at request creation from the same four severity factors (`core/utils/dispatch.py`). `derive_priority_tier()` returns a discrete label for the critical-bypass decision and the operator badge, and `compute_priority_lambda()` returns a continuous score in $[0, 1]$ that weights the Hungarian cost matrix. Critical-tier requests bypass Hungarian via KNN, so only standard- and elevated-tier requests are ever weighted.

Table 10 - Appendix Table 5 - Priority and weighting infrastructure definitions.

Parameter	Value	Notes
Raw factor contributions	medical 0.4 / hazardous 0.3 / children 0.2 / alone 0.1	Sum to exactly 1.0 when every factor is true, which is what makes 1.0 the natural ceiling for lambda rather than an arbitrary cap.
CRITICAL_CEILING	6.0	<code>weight_from_lambda(1.0) = 6.0</code> ; the upper bound a request's distance can be amplified by inside the cost matrix.

Weight formula	$\text{weight} = 1.0 + \text{lambda} \times (\text{CRITICAL_CEILING} - 1.0)$	Straight-line interpolation between 1.0 (lambda = 0) and CRITICAL_CEILING (lambda = 1).
Tier rule	medical → critical; hazardous + (children or alone) → critical; hazardous or children alone → elevated; else standard	A discrete yes/no decision, used only for the critical-bypass and the operator badge.
Critical-tier requests	Bypass Hungarian; dispatched via Mode 1 (KNN) directly	Never enter assign_multiple_requests(), so lambda values near 1.0 are structurally unreachable inside the weighted cost matrix - the recalibration referred to in Section 5.3.

E.2 The continuous score distinguishes same-tier requests

Every combination of the four severity factors was enumerated by calling `derive_priority_tier()` and `compute_priority_lambda()` directly. “Medical concern only” and “medical + hazardous + children” are both classified critical, but the continuous score separates them (0.40 vs 0.90), which is the concrete demonstration referenced in Section 5.3:

Table 11 - Appendix Table 6 - Continuous score definitions. Computed live from `core/utils/dispatch.py` (`derive_priority_tier`, `compute_priority_lambda`, `weight_from_lambda`).

Scenario	Tier	Lambda	Weight
No risk factors	standard	0.00	1.000
Alone, no other factors	standard	0.10	1.500
Hazardous location only	elevated	0.30	2.500
Children onboard only	elevated	0.20	2.000
Hazardous + children	critical	0.50	3.500
Hazardous + alone	critical	0.40	3.000
Medical concern only	critical	0.40	3.000
Medical + hazardous + children	critical	0.90	5.500

E.3 Amplifying versus discounting: a deliberate tie

A two-request, two-operator scenario was constructed with real Addis Ababa coordinates so that, unweighted, the near operator goes to the standard request rather than the higher-priority one: request P (hazardous location, $\lambda = 0.30$, elevated tier) is 2.002 km from the near operator and 12.658 km from the far one; request S (standard) is 1.001 km from the near operator and 11.976 km from the far one. Without any weighting, Hungarian assigns the near operator to S, not P. The scenario was then run three ways, changing only how P's row is weighted:

Table 12 - Appendix Table 7 - Amplification vs discounting

Weighting	P (priority) assigned to	S (standard) assigned to
Unweighted (weight = 1.0)	Far operator - 12.658 km	Near operator - 1.001 km
Amplify - production weighting (CRITICAL_CEILING = 6.0, weight = 2.500)	Near operator - 2.002 km	Far operator - 11.976 km
Discount - rejected theory (weight = 0.750)	Far operator - 12.658 km	Near operator - 1.001 km

The production weighting is the only one of the three that gives the higher priority request the near operator; the discount variant reproduces the unweighted outcome exactly, which is the direct computation behind the code comment in `dispatch.py` explaining why an earlier discounting version of the weight was rejected: “discounting a row makes the solver less careful about where that row ends up, protecting the undiscounted row instead.” All three assignments were computed by calling the real `assign_multiple_requests()`, not derived by hand; the discount case was produced by temporarily setting `CRITICAL_CEILING` to `1/6` so that `weight_from_lambda(0.30)` evaluates to `0.750`, the reciprocal-scaled analogue of the production ceiling.

Computed live from `core/utils/dispatch.py`.

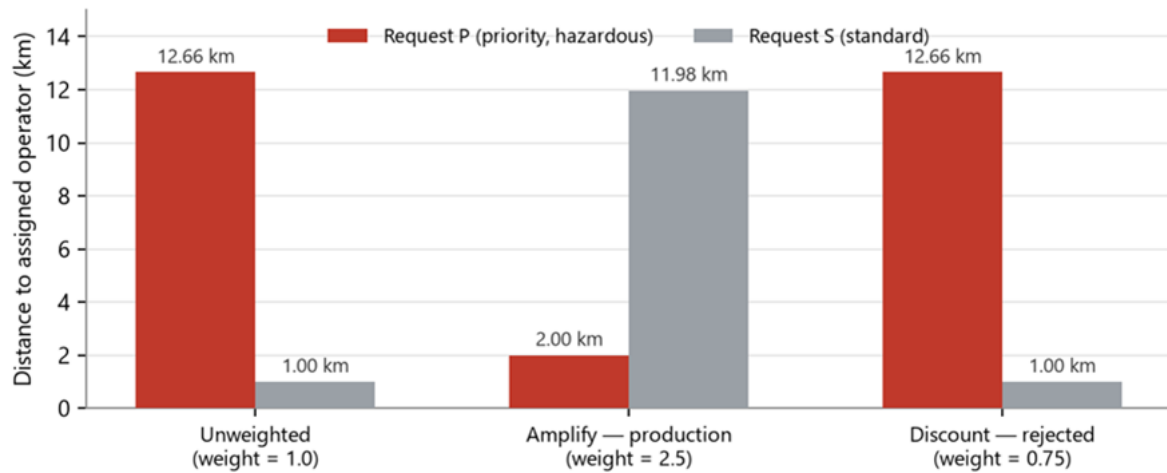


Figure 12 - Appendix Figure 1 - which operator each request is assigned to, by weighting regime. Request P (red) is stuck with the far operator under both the unweighted baseline and the rejected discount theory. Only the production amplify weighting flips it to the near operator.

E.4 Automated test-suite output (test_priority_tier_integration.py)

test_priority_tier_integration.py

(backend/depannini/core/tests/test_priority_tier_integration.py) exercises the full HTTP view stack rather than the algorithm directly: it posts two real requests concurrently through AssistanceRequestView, confirms priority_tier is derived and saved for both the towing and repair branches, and confirms a critical request wins a contested operator over a standard one in the same batch. Run against PostgreSQL, all three tests pass, including the concurrent-batching case, which depends on row-level locking that only PostgreSQL enforces:

Table 13 - Appendix Table 8 - Automated testing outputs. Summary of test_priority_tier_integration.py's three tests.

Test	What it checks	Result
test_critical_request_gets_close_operator...	A critical request wins a contested operator via the real concurrent HTTP+batch path	PASS - resolved under PostgreSQL
test_priority_tier_is_stored_correctly_for...	priority_tier is derived and saved correctly (repair branch)	PASS
test_no_risk_factors_stores_standard_tier	No risk factors → standard tier, no	PASS

	crash on missing keys	
--	-----------------------	--

Full verbose output follows below.

```

Creating test database for alias 'default' ('test_depannini_test')...
Found 3 test(s).
Operations to perform:
  Synchronize unmigrated apps: channels, messages, rest_framework,
rest_framework_simplejwt, staticfiles
  Apply all migrations: admin, auth, contenttypes, core, sessions
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  [... 32 migrations applied, output omitted for brevity ...]
test_critical_request_gets_close_operator_when_batched_with_standard
(core.tests.test_priority_tier_integration.PriorityTierEndToEndTests.test_critical_req
uest_gets_close_operator_when_batched_with_standard) ... ok
test_no_risk_factors_stores_standard_tier
(core.tests.test_priority_tier_integration.PriorityTierEndToEndTests.test_no_risk_fact
ors_stores_standard_tier) ... ok
test_priority_tier_is_stored_correctly_for_repair_requests
(core.tests.test_priority_tier_integration.PriorityTierEndToEndTests.test_priority_tier
_is_stored_correctly_for_repair_requests) ... ok

-----
Ran 3 tests in 6.684s

OK
Destroying test database for alias 'default' ('test_depannini_test')...
OK
System check identified no issues (0 silenced).

```

test_priority_tier_integration.py - full verbose output on PostgreSQL, 3/3 passed.

E.5 Resilience and concurrency

test_concurrent_dispatch_stress.py

(backend/depannini/core/tests/test_concurrent_dispatch_stress.py) is the dedicated stress test for the claim that `select_for_update()` plus `transaction.atomic()` prevents two concurrent dispatch passes from ever assigning the same operator to two different requests. It fires many more requests than there are operators from real concurrent threads at the live HTTP endpoint, across several rounds, run against a real PostgreSQL

database provisioned through Docker, the backend that actually enforces row-level locking under concurrent writes.

Across 75 concurrent requests over five rounds, the test confirms that no operator is ever assigned to more than one request at the same time:

```
Creating test database for alias 'default' ('test_depannini_test')...
Found 1 test(s).
Operations to perform:
  Synchronize unmigrated apps: channels, messages, rest_framework,
rest_framework_simplejwt, staticfiles
  Apply all migrations: admin, auth, contenttypes, core, sessions
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  [... 32 migrations applied, output omitted for brevity ...]
test_no_operator_is_ever_double_assigned_under_concurrent_load
(core.tests.test_concurrent_dispatch_stress.ConcurrentDispatchStressTests.test_n
o_operator_is_ever_double_assigned_under_concurrent_load) ...
  [... per-round HTTP request/response and photo/voice-note debug logging omitted
for brevity ...]
ok

-----

Ran 1 test in 45.873s

OK
Destroying test database for alias 'default' ('test_depannini_test')...
OK
System check identified no issues (0 silenced).
```

PASS: the real test PASSES against the live code - 75 concurrent requests, zero double-assignments

The result was reproduced independently on three separate runs against fresh PostgreSQL instances (45.9s, 55.2s and 35.0s), including inside `Concurrency_Safety_Bug_And_Fix.ipynb`, which holds the full record.

E.6 Cascade verified: a released request re-enters the pool, not a fixed ranked list

Two scenarios confirm this against the live code and PostgreSQL: the simple case of a single decline with one remaining operator, and the case that motivates batching the

cascade at all, two requests timing out at once. In the second scenario R1 is only mildly closer to the near operator while R2 is overwhelmingly closer to it and far worse off with the alternative, so a fixed, order-dependent re-offer would hand the near operator to whichever request happened to be processed first.

Full verbose output follows below.

```
Creating test database for alias 'default' ('test_depannini_test')...
Found 2 test(s).
Operations to perform:
[... 32 migrations applied, output omitted for brevity ...]
test_declined_request_is_released_to_the_pool_not_lost ... ok
test_simultaneous_timeouts_are_batched_not_cascaded_one_at_a_time ... ok

-----

Ran 2 tests in 2.914s

OK
Destroying test database for alias 'default' ('test_depannini_test')...
OK
System check identified no issues (0 silenced).

--- 1. A declined request is released back to the pending pool ---
response: 200 {'status': 'cascaded', 'new_operator_id': 2}
assistance.status: requested
assistance.declined_operator_ids: [1]
assistance.assistant_id: 2
PASS: request stayed 'requested' (never lost) and was re-dispatched to the backup
operator

--- 2. Two simultaneous timeouts are resolved jointly via Hungarian, not one-at-a-
time ---
Released 2 stale offer(s); swept 2 pending request(s).
R1 -> OpFar
R2 -> OpNear
PASS: both released, timed-out-operator excluded, jointly re-assigned via one
shared batch sweep, not a fixed ranked re-offer
```

Cascade release-and-rebatch verification, run against PostgreSQL - 2/2 passed. Both requests correctly recorded the declined or timed-out operator, so it is never re-offered the same request while remaining fully eligible for any other pending request.